Contents

1 Introduction **1**
- 1.1 Proposed Solutions . . . 1
- 1.2 Motivation . . . 2

2 Background **3**
- 2.1 Prediction Methods in Credit Card Fraud Detection . . . 4
- 2.2 Feature Processing and Enhancement Methods . . . 6

3 Project Execution **8**
- 3.1 The Proposed Approaches . . . 8
- 3.2 Problem Formalization . . . 8
- 3.3 Model Architecture . . . 9
- 3.4 Loss Optimization . . . 16
- 3.5 Prevent Label Information Leakage . . . 17
- 3.6 Model Evaluation Method . . . 18

4 Critical Evaluation **19**
- 4.1 Datasets . . . 19
- 4.2 Experiment Settings . . . 20
- 4.3 Detection Performance . . . 21
- 4.4 Training Performance Analysis Across Datasets . . . 23
- 4.5 Performance Comparison of Feature Processing Models . . . 25
- 4.6 K-Fold Cross-Validation Analysis . . . 25
- 4.7 Correlation Analysis of Time Series Features . . . 26
- 4.8 Comparison of Distribution Methods in Ma-CVAE . . . 27
- 4.9 Ablation Study . . . 28

5 Conclusion **29**
- 5.1 Contributions and Achievements . . . 29
- 5.2 Future Work . . . 30

A Appendix **36**
- A.1 Dynamic-MLP . . . 36
- A.2 Comparison with Baseline GCN . . . 36

List of Figures

1.1 Illustration of Feature Generalization and Diversity Enhancement after Ma-CVAE Processing. The input features are first projected into a 2D space using PCA for visualization. Subsequently, a Density-Based Spatial Clustering of Applications with Noise (DBSCAN)[19] is applied to identify clusters by finding regions of high density. The left figure demonstrates that after Ma-CVAE processing, the features exhibit more diverse clustering compared to the single cluster observed in the features before Ma-CVAE processing. 2

2.1 The illustration of the framework for the CCFD process. First, when a transaction request is received by the user's account, basic checks are performed, including verifying the correct PIN entry and ensuring sufficient account balance. Then, the transactions that pass these checks are subjected to risk prediction using a predictive model, which generates Fraud Scores. Finally, financial experts manually review transactions with high fraud risk, making decisions on whether the transactions are fraudulent or legitimate based on the audit results. 3

3.1 The illustration of the overview of the adaptivae semi-supervied method for CCFD. The task encompasses three primary components: the Feature Preprocessor, Ma-CVAE, and TGAT. It starts with the input of Numerical attributes X_{na}, Categorical attributes X_{ca}, and Risk information X_r. The Ma-CVAE then processes these inputs, using three types of labels as conditions to guide the learning of latent feature diversities. Following this, the TGAT[69] is employed to learn the spatiotemporal feature representations. Finally, an MLP layer outputs the scores for CCFD. 8

3.2 The illustration of the random label masking process. Based on the original labels, some of the labels are randomly masked and set as unknown, represented by label 2. As shown in the scatter plot, the green points represent the unknown new label obtained by randomly masking the original purple and red labels. This process is used to construct a semi-supervised learning dataset. 10

3.3 The illustration of the Multi-Head Attention mechanism following the self-attention mechanism [65]. In this mechanism, the input label features are transformed into queries, keys, and values. The similarity between the queries and all keys is calculated, resulting in a set of weights through a Softmax layer. These weights are then used to compute a weighted sum of the corresponding values, yielding the final value in the attention mechanism. . . . 11

3.4 An illustration of the Ma-CVAE model, demonstrating the two approaches used to map data through different distributions: Gaussian distribution and Gumbel-Softmax distribution [33]. These methods are used to learn the latent representations of the probabilistic distribution of features while generating reconstructed features that are similar to the original data under the supervision of label information(condition), the label information, which serves as the condition in this context. 13

3.5 Cited from Jang et al. [33], this figure illustrates the sample distribution results for categorical features under different values of τ. As shown in the figure, when $\tau = 10$, the distribution approaches a uniform distribution, while when τ is very small, such as 0.1, it focuses on one specific category feature. In the CCFD task, specifically in the FFSD dataset, there are categorical and numerical features, which are combined for feature learning. However, the Gaussian distribution sampling process cannot directly achieve this, potentially leading to gradient issues during backpropagation. Therefore, Gumbel distribution sampling is used to balance the sampling process of discrete and continuous features, and a relatively large value of τ is chosen to make the distribution more uniform. 13

3.6 This illustration demonstrates the working principle of the TGAT model, which applies a Temporal Transaction Graph based on the Graph Attention Networks (GAT) [66]. The figure is composed of three subfigures: (a) shows the Temporal Transaction Graph obtained through data processing; (b) illustrates how each node in the Temporal Transaction Graph calculates the Attention by applying learned weight parameters to the features of both its neighbors and itself, followed by a softmax activation function; (c) depicts the multi-head attention mechanism, where each color represents a different attention head. Compared to a single attention mechanism, the multi-head attention mechanism is better at learning the latent representations of features. 14

4.1 The illustration of the FFSD, Amazon, and Yelp datasets, where red represents records, blue represents legitimate records, green represents fraudulent records, and orange represents unlabelled records. These different categories are manually labeled. Due to the significantly larger number of records in the FFSD dataset compared to Amazon and Yelp, the cost of manual labeling increases substantially, resulting in a large number of unlabelled records. 19

4.2 Illustration of the training and validation losses across different epochs for the FFSD [69], Amazon [34], and Yelp [49] datasets. The blue, green, and purple lines represent the training loss trends for the three datasets, showing a decreasing pattern as the number of epochs increases, indicating that the model continues to learn and adjust its parameters. The orange, brown, and red lines represent the validation losses. Unlike the training set, the validation set is unseen during training and is used to evaluate the model's performance on unseen data. For the Amazon dataset, an overfitting trend is observed as the number of epochs increases, which is why early stopping [45] is employed. 24

4.3 Illustration of the training performance using three metrics across 15 epochs for the FFSD, Amazon, and Yelp datasets. The cyan, blue, and red lines represent the trends of AUC, AP, and Accuracy for the three datasets. 25

4.4 An illustration of the K-fold cross-validation process for the FFSD, Amazon, and Yelp datasets, showing the performance of the models on the validation set. To mitigate the instability in model evaluation caused by the randomness of the training data sampling, a smaller K value (K=5) is used, ensuring that each sample is utilized for both training and validation. Additionally, the validation losses across different folds demonstrate the specific performance variations across these folds. 26

4.5 An illustration of the correlation matrix of historical time series features across three datasets: FFSD, Amazon, and Yelp. The time series features with high correlation are highlighted in red boxes. It can be observed from the figure that the deeper the green color, the higher the correlation between different time features. Apart from the diagonal, where the correlation between each feature and itself is the highest, there are highly correlated regions around key transaction features, forming clusters of high correlation. These clusters are useful for studying the correlation among time features. Additionally, the overall color intensity variations across different datasets reflect the differences in the distribution of features in these datasets. 27

5.1 Illustration of the complementary attention weight matrix. The complementary attention mechanism consists of Normal Attention and Inverse Attention, where the value of Attention is denoted as $\mathbf{A}$, and the value of inverse Attention can be expressed as $1 - \mathbf{A}$. A learnable weight $\mathbf{W_a}$ is used to balance these two types of Attention, resulting in a reasonable weight distribution. 30

A.1 An illustration of the dynamic MLP structure. This structure maps the given input matrix with dimensions $\mathbb{R}^{N \times d}$ to a weight matrix with dimensions $\mathbb{R}^{N \times N}$, where N represents the temporal dimension. The resulting temporal matrix captures the feature correlations along the temporal dimension. This feature correlation is then multiplied with the original input matrix to obtain a new matrix that integrates temporal feature correlations. 36

List of Tables

3.1 Given a time window T, the data for each transaction and account type is aggregated and further processed to derive eight new features. The table includes descriptions of each feature type for CCFD tasks. 9

4.1 Hyperparameter Settings in the Experiments 20
4.2 Comparison of the performance on the FFSD dataset using AUC, F1, and AP metrics with the SOTA model TGAT[69] and other advanced models. The best performance values are highlighted in bold, and the variance is used to indicate the error in performance. 21
4.3 This table presents a comparison of various models' performance on the Amazon review dataset, evaluated using AUC, F1, and AP metrics. The TGAT[69] model, recognized as the SOTA, along with other advanced models, is included in this comparison. The best-performing results are emphasized in bold, and variance is provided to represent the performance error. 22
4.4 Comparison of model performance on the YelpChi dataset, evaluated using AUC, F1, and AP metrics. The highest performance values are highlighted in bold, with the corresponding variance indicating the margin of error. 22
4.5 Comparison of TGAT model performance across different layers (L = 1 to L = 5) on the FSSD dataset, evaluated using AUC, F1, and AP metrics. The best-performing results are highlighted in bold. The comparison between L = 2 without Ma-CVAE and L = 4 with Ma-CVAE demonstrates that incorporating Ma-CVAE allows the optimal number of layers to increase, improving overall performance and reducing oversmoothing. 23
4.6 Comparison of the proposed Ma-CVAE model with other feature processing models, such as the generative model VAE[36] and the diffusion model Con-Diffusion[73], across three datasets: FSSD, Amazon, and YelpChi. The evaluation metrics used are AUC, F1, and AP, with the best values highlighted in bold. 26
4.7 Comparison of the performance on the YelpChi dataset without Ma-CVAE and with Ma-CVAE using two different sampling methods: Gumbel [33] and Gaussian distributions. The evaluation is based on three metrics: AUC, F1, and AP, with the best-performing results highlighted in bold. 28
4.8 Comparison of the performance across FSSD, Amazon, and YelpChi datasets without Ma-CVAE (w/o) and with Ma-CVAE (w). The evaluation is based on three metrics: AUC, F1, and AP, with the best-performing results highlighted in bold. 28

A.1 Comparison of the proposed Ma-CVAE + TGAT model with the traditional Baseline GCN [41] across the Amazon and YelpChi datasets based on two metrics: AUC and F1. The best-performing results are highlighted in bold. 37

Ethics Statement

This project fits within the scope of the blanket ethics application id 15208, as reviewed by my supervisor Yan Ge

Chapter 1

Introduction

In recent years, the expansion of online commerce has dramatically changed the landscape of commodity trading. According to Visa and Mastercard, 2287 million credit cards were issued in a single quarter [8], highlighting the global adoption of credit card payments as a preferred method [68, 75], which has significantly enhanced transaction efficiency. Consequently, an increasing number of predictive models are being developed to automatically detect fraud patterns and generate a fraud risk score, which is then used to determine whether the target transaction is fraudulent. Essentially, the predictive model functions as a binary classification model. Once the model produces its results, financial experts can efficiently focus on high-risk transactions and feed their analysis back into the predictive model for continuous improvement.

Over the past 20 years, traditional machine learning methods such as Markov models [60], SVM [53], Random Forest [70], and XGBoost [64, 32] have been widely used in CCFD tasks. However, these traditional machine learning methods are overly dependent on specific formats of datasets and rely on manually designed and extracted features, which require domain expertise and a significant amount of time and are not robust [6]. As a result, in recent years, deep learning methods have become increasingly popular, with models like LSTM [35], CNN [11, 46], Transformer [5], and some generative models such as GAN [21, 31] being used to address issues like data imbalance. However, these methods often overlook the relational dependencies between transactions, leading to a disjointed learning process when distinguishing features of different label types, whereas CCFD tasks heavily rely on such relational information. Therefore, currently, GNN-based methods have been proposed to utilize the relational information between different transactions as a means to supervise model learning and have achieved promising results [69, 13, 41]. However, some research methods have found that GNN-based methods, when stacking layers, may experience performance degradation in deep GNNs due to over-smoothing, particularly during the homogenization of a large number of dataset labels [52, 23, 40, 56]. In anomaly detection tasks like CCFD, the issue of data imbalance exists, making GNN-based models prone to over-focusing on features of the majority class as the model layers deepen, which can lead to the smoothing out of distinctions between fraudulent and legitimate features.

1.1 Proposed Solutions

This paper proposes a Multi-head Attention-Conditional Variational Autoencoder model. This model combines numerical features, categorical features, and transaction-related features, utilizing a multi-head attention mechanism to focus on the characteristics of fraudulent transactions and their neighboring transactions. The features are then mapped into the Gumbel-Softmax space [33] for feature diffusion. This approach simulates the evolving strategies of fraud techniques, increasing feature diversity and thereby enhancing the model's sensitivity to various fraudulent feature patterns. Additionally, it mitigates the issues of over-smoothing and over-squashing that are prevalent in GNN-based models[69, 16, 13, 42, 13]. Furthermore, due to the large volume of data in real-time datasets that require confirmation by cardholders and risk assessment by experts to determine risk labels—an effort that consumes significant human and time resources [69, 67]—the majority of real-time datasets are actually unlabeled. Therefore, to simulate and construct a semi-supervised dataset, some of the feature labels will be randomly masked, and these masked features will serve as input to the Ma-CVAE model. The reconstructed features output by the Ma-CVAE will then be fed into a Gated Temporal Attention Network [69], which is a predictor model designed to learn the spatiotemporal characteristics of historical transaction information. Finally, a two-layer MLP maps the results, yielding the transaction risk score.

In Figure 1.1, to qualitatively describe the changes in feature diversity, the method proposed by Tang et al. [62, 61] was followed to visualize feature diversity using clustering. Specifically, features before and after processing by Ma-CVAE are first mapped into a 2-dimensional space using PCA for visualization purposes. Then, DBSCAN [19] is used to cluster the features before and after inputting into Ma-CVAE. Since DBSCAN divides different clusters based on feature similarity, the presence of more clusters with the same number of features indicates greater feature diversity. Thus, it can be observed that, under the same parameter settings, the features processed by Ma-CVAE exhibit a higher number of clusters (5 clusters) in the left image, compared to the unprocessed features in the right image (1 cluster), indicating greater diversity and a higher degree of diffusion.

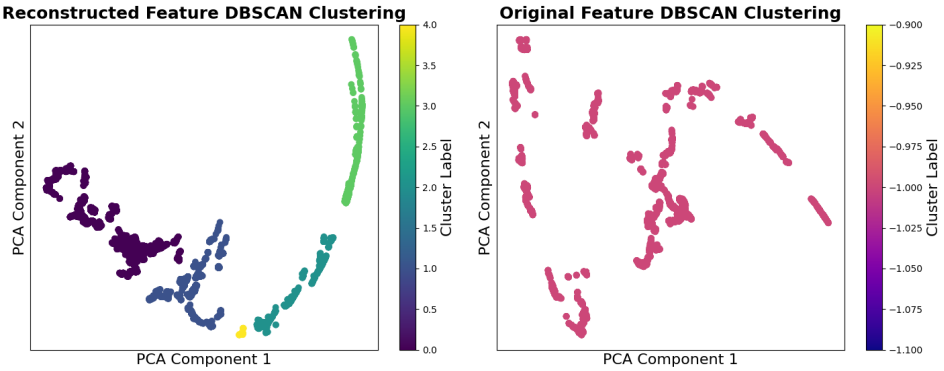


Figure 1.1: Illustration of Feature Generalization and Diversity Enhancement after Ma-CVAE Processing. The input features are first projected into a 2D space using PCA for visualization. Subsequently, a Density-Based Spatial Clustering of Applications with Noise (DBSCAN)[19] is applied to identify clusters by finding regions of high density. The left figure demonstrates that after Ma-CVAE processing, the features exhibit more diverse clustering compared to the single cluster observed in the features before Ma-CVAE processing.

1.2 Motivation

The current state-of-the-art approach [69] and other advanced fraud detection methods [13, 41, 17, 43, 67] leverage sophisticated graph embedding techniques, such as node2vec or inherent neighbor information from datasets, along with real-world transaction attributes. These features are combined, and labels are strategically masked in a randomized manner to simulate a semi-supervised learning process, reducing the risk of overfitting. However, this method has not fundamentally improved the distinguishability of different types of features. The Ma-CVAE model proposed in this paper effectively addresses these issues. Specifically, the contributions of this paper are outlined as follows:

1. The existing approaches do not adequately enhance feature diversity during model training, which can lead to feature overfitting. This issue becomes more pronounced as fraud techniques evolve, causing a significant decline in the model's ability to detect fraud when encountering novel features that were absent in the training data. Our proposed model effectively mitigates this problem.

2. By employing a multi-head self-attention mechanism on label features, our approach more effectively handles the distribution of weights among the three types of semi-supervised labels (Legitimate, fraud, Unknown) during the learning phase. This method surpasses the traditional approach of merely combining labels with features, thereby fully exploiting the potential of semi-supervised learning.

3. By mapping features into the Gumbel-Softmax space, the proposed model enhances sensitivity to feature variations. This approach addresses the common issues of over-smoothing and over-squashing that arise in GNN-based models, particularly when dealing with imbalanced datasets and increasing the number of layers.

4. The proposed model is evaluated on the financial dataset FFSD as well as two fraud detection datasets, Amazon and YelpChi. Comparative analysis against other state-of-the-art methods across various metrics reveals that our model achieves superior performance.

Chapter 2

Background

Credit card fraud detection (CCFD) is a prominent task within the broader research field of fraud detection task, as it represents one of the main methods used in illegal financial activities, such as online scams, leading to significant financial losses [2, 44]. Credit card fraud involves the unauthorized use of funds in transactions, typically conducted through credit or debit cards. This type of fraud not only results in direct financial losses for cardholders but also poses significant risks to financial institutions and payment networks, such as money laundering and further fraudulent activities [7]. As a result, the task of identifying and preventing credit card fraud has gained increasing importance since the early 21st century, particularly with the rapid growth of online transactions. Numerous approaches have been proposed, focusing mainly on enhancing predictive models, as shown in Figure 2.1. Unlike traditional predictive models, which rely on fixed rule-based mechanisms to assess transaction risks, these new methods offer more dynamic and adaptable solutions. Traditional models can often be bypassed by fraudsters who learn to exploit these fixed rules [69, 14]. The current mainstream approach involves the development of automated detection models rather than relying on fixed local rules. The detailed methodologies for these models are discussed in Section 2.1.

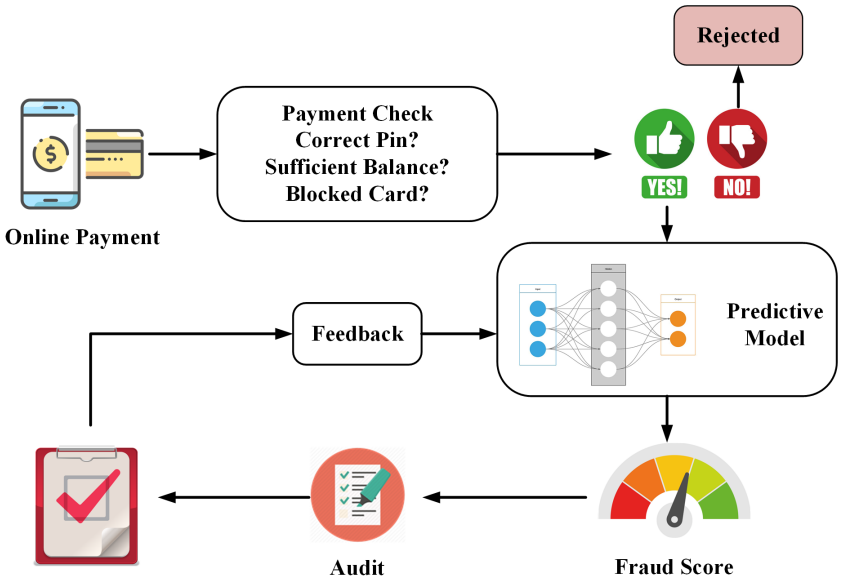


Figure 2.1: The illustration of the framework for the CCFD process. First, when a transaction request is received by the user's account, basic checks are performed, including verifying the correct PIN entry and ensuring sufficient account balance. Then, the transactions that pass these checks are subjected to risk prediction using a predictive model, which generates Fraud Scores. Finally, financial experts manually review transactions with high fraud risk, making decisions on whether the transactions are fraudulent or legitimate based on the audit results.

2.1 Prediction Methods in Credit Card Fraud Detection

As a subset of fraud detection tasks, CCFD essentially uses binary classification models to determine whether a given transaction record is high-risk or fraudulent based on historical transaction data. Earlier methods primarily relied on rules-based approaches [55], where financial experts would establish detailed transaction risk rules to identify high-risk transactions. Nowadays, mainstream methods are categorized into traditional machine learning methods [60, 53, 70, 64, 32] and deep learning approaches [69, 13, 14, 22, 20], each with its own advantages, which will be discussed in detail in the following sections.

2.1.1 Traditional Machine Learning Methods

HMM: Srivastava et al. [60] proposed a Markov model approach applied to the CCFD task, utilizing a Hidden Markov Model (HMM) [4] to simulate the sequence of operations in credit card transaction processing. By learning the historical data of the cardholder's normal spending behavior, the model can distinguish between legitimate transactions and potential fraudulent ones.

In an HMM, the sequence of credit card transactions is modeled as a process of state transitions. The hidden states are represented by the set $\mathcal{S} = \{s_1, s_2, \ldots, s_N\}$, where each state s_i corresponds to a specific spending behavior pattern of the cardholder. The transaction sequence $\mathcal{R} = \{r_1, r_2, \ldots, r_T\}$ represents the observed transaction records, with r_t denoting the transaction at time t. Given a new sequence of credit card transactions $\mathcal{R}$, the probability of generating this sequence $P(\mathcal{R}|\lambda)$ can be calculated, where λ represents the model's parameters. If the calculated probability $P(\mathcal{R}|\lambda)$ for a given transaction sequence $\mathcal{R}$ falls below a certain threshold, the transaction is considered potentially fraudulent.

While the HMM [4] model can effectively model and infer changes in transaction sequences, it heavily depends on the accuracy of the training data. If there is a sudden change in the cardholder's behavior, such as an abrupt shift in spending habits, the model may struggle to accurately detect fraud, potentially leading to an increase in false positives or negatives.

XGBoost: In CCFD tasks, fraudulent transactions are typically the minority, and some machine learning methods, such as HMM [4] and Bayesian models, struggle to effectively learn from such imbalanced data. These methods often overlook the characteristics of minority class samples with fraud labels, leading to poor differentiation of fraudulent transactions. To address this issue, Noviandy et al. and Ileberi et al. [47, 32] proposed the use of XGBoost [12] for feature learning in CCFD tasks. On the one hand, XGBoost [12] allows users to assign different weights to samples of different classes. By assigning higher weights to the minority class samples (fraudulent transactions), the model gives these samples greater influence during training. On the other hand, XGBoost [12] iteratively updates the model by minimizing the classification error in each round. This process focuses particularly on those samples that are difficult to classify. In the context of CCFD tasks, minority class samples (fraudulent transactions) are inherently harder to distinguish due to their limited data characteristics available for the model to learn from. As shown in Eq. 2.1, the residual $r_i^{(t)}$ for sample i in the t-th iteration reflects the prediction error from the previous iteration, which is the difference between the predicted and actual values. XGBoost [12] then focuses more on these challenging samples in subsequent iterations.

$$r_i^{(t)} = -\frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}} \tag{2.1}$$

However, while XGBoost [12] can mitigate the problem of imbalanced data to some extent by assigning higher weights to minority class samples during training, this approach is not adaptive. The effectiveness of this method heavily relies on the precise tuning of model hyperparameters due to XGBoost's sensitivity to these parameters. If the hyperparameters are not optimally tuned, the model may fail to achieve the expected performance.

2.1.2 Deep Learning Methods

LSTM-based: In CCFD tasks, using historical transaction records to predict or classify the risk of a target transaction imposes high demands on the model's ability to learn temporal dependencies in the data. However, traditional machine learning methods like HMM [4] face challenges when dealing with long-term dependencies in historical transaction features because they rely on the Markov property. Jurgovsky et al. [35] proposed using Long Short-Term Memory (LSTM) networks to learn temporal dependencies through a gating mechanism. Specifically, the forget gate in LSTM [4] determines which

historical transaction features are important for the risk prediction learning process in CCFD tasks. The relevant features are retained, while less important ones are forgotten, as shown in Eq. 2.2. The forget gate output f_t ranges from 0 to 1, indicating the degree to which each component of the cell state is forgotten, effectively filtering the information and addressing the vanishing gradient problem commonly encountered in Recurrent Neural Networks (RNNs) [51] models.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{2.2}$$

However, while LSTM [4] is highly effective in capturing the long- and short-term dependencies of certain time periods in historical transactions for CCFD tasks, it is less sensitive to the importance of learning internal features within individual transactions. Additionally, LSTM [4] struggles with learning the correlations between different features in each transaction, which can limit its effectiveness in fully understanding the transactional data.

CNN-based: As previously mentioned, LSTM is less sensitive to the relationships between features, particularly when dealing with local feature information. Therefore, Fu et al. [22] proposed using Convolutional Neural Networks (CNNs) [39] for CCFD tasks. Unlike methods such as XGBoost [12], which involve manually adjusting feature learning weights, CNNs [39] can automatically extract features from raw data. CNNs [39] are particularly suited for learning the underlying relationships between features. In CCFD tasks, certain fraud transactions often lack obvious feature relationships that directly indicate high-risk transactions. In such cases, CNNs [39] transform the transaction features into high-dimensional feature matrices through convolution operations, enabling the model to learn the underlying relationships and complex combinations between transaction features.

As shown in the equation below, the input transaction data, represented as a feature vector $\mathbf{x}$, is transformed into a high-dimensional feature matrix $\mathbf{F}$ through convolution operations. For a given convolutional filter $\mathbf{W}$ and bias b, the convolution operation can be shown as:

$$\mathbf{F}_{i,j} = \sigma\left(\sum_{m=1}^{M}\sum_{n=1}^{N}\mathbf{W}_{m,n} \cdot \mathbf{x}_{i+m-1,j+n-1} + b\right) \tag{2.3}$$

While CNNs [39] excel at learning complex combinations of local features, their primary strength lies in spatial feature learning. In the context of CCFD tasks, however, there is not only a need to learn the relationships between different transactions over time (temporal dependencies) but also the relationships between features within each transaction. Balancing the learning process of both spatial and temporal features remains a challenge for CNNs [39], as they are not inherently designed to handle temporal sequences as effectively as they handle spatial features.

GNN-based: In the CCFD task, each transaction can be viewed as a node, with information such as account details, merchants, and transaction types represented as node features. The relationships between different account transactions can be learned through the edges connecting these nodes. Fraudulent transactions often involve multiple nodes and multi-layer relationships, making Graph Neural Networks (GNNs) particularly suited for capturing both global and local information within the graph structure. By incorporating temporal graphs, GNNs can also learn the temporal features of the transaction sequences. For example, Xu et al. [57] and Liu et al. [41] have applied GNN-based models in fraud detection. However, traditional GNN models [54] aggregate information from neighboring nodes using fixed parameters or aggregation functions, such as sum, mean, or max, as shown in Eq. 2.4. These fixed aggregation methods are not well-suited for adaptively learning the importance of information from neighboring nodes.

$$\mathbf{h}_i^{(k)} = \sigma\left(\mathbf{W}^{(k)} \cdot \text{AGGREGATE}\left(\left\{\mathbf{h}_j^{(k-1)} : j \in \mathcal{N}(i)\right\}\right) + \mathbf{b}^{(k)}\right) \tag{2.4}$$

Subsequently, the Graph Convolutional Network (GCN) model [38] improved upon the traditional GNN by introducing a normalization factor $\frac{1}{\sqrt{d_i d_j}}$ that weights the features of neighboring nodes during aggregation, as shown in Eq. 2.5.

$$\mathbf{h}_i^{(k)} = \sigma\left(\sum_{j \in \mathcal{N}(i)\cup\{i\}} \frac{1}{\sqrt{d_i d_j}}\mathbf{W}^{(k)}\mathbf{h}_j^{(k-1)}\right) \tag{2.5}$$

While this approach alleviates the problem of over-reliance on certain nodes during feature aggregation, it may weaken the model's ability to learn critical features from key nodes, such as those representing fraudulent transactions. This can lead to a decrease in the accuracy of transaction risk prediction, especially when dealing with dominant fraudulent transaction nodes. To address this issue, Petar et al. [66] proposed the Graph Attention Networks (GAT), which introduces an attention mechanism during the aggregation process, as described in Eq. 2.6. The attention weights $\alpha_{ij}^{(k)}$ are adaptively learned during the training process, allowing the model to focus on important transaction nodes that significantly impact the outcome.

$$\mathbf{h}_i^{(k)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(k)} \mathbf{W}^{(k)} \mathbf{h}_j^{(k-1)}\right) \tag{2.6}$$

In the CCFD task, a state-of-the-art model proposed by Xiang et al. [69] is based on the GAT model and has achieved outstanding results. In this work, following their model setup, a temporal graph is input into the GAT model, enabling the model to adaptively learn both the neighboring transaction nodes and the temporal dynamics of these transactions.

2.2 Feature Processing and Enhancement Methods

Similar to most fraud detection tasks, CCFD encounters the challenge of dealing with fraudulent transactions, which are labeled as anomalies and represent only a small fraction of the overall transaction volume [15, 10]. This data imbalance presents a significant challenge for the model to learn the characteristics of fraudulent transactions and accurately predict the associated risks. The learning process for such minority samples is prone to issues like overfitting. Additionally, as noted by Shi et al.[52] and Giraldo et al.[23], when using GNN-based models [69, 13, 41, 17, 67], increasing the number of layers can reduce the distinguishability between feature nodes. As a result, the features of nodes gradually blend with those of their neighbors, ultimately leading to feature homogenization, where all node features become similar. This phenomenon, known as oversmoothing, particularly affects the model's performance on datasets with few samples. To address the challenges posed by data imbalance, a series of Feature Processing and Enhancement Methods have been proposed. These methods aim to improve the model's ability to learn from minority samples and enhance the overall feature representation. In the following sections, some oversampling methods and generative models will be employed to enhance the ability to learn from minority samples and improve overall feature representation. These methods will be discussed in detail.

Oversampling Methods: In tasks that frequently encounter data scarcity, such as time series forecasting [30], it is common to use resampling or oversampling techniques for minority samples. These methods help maintain data continuity and enhance the effectiveness of the dataset. Similarly, these approaches have been applied to address data imbalance problems and have shown promising results [9, 27, 26]. Chawla et al. [9] introduced the Synthetic Minority Over-sampling Technique (SMOTE), which generates new synthetic samples by interpolating between existing minority class samples. This method has been effectively used to address the sample imbalance in CCFD tasks. For instance, Ileberi et al. [32] applied this approach to credit card transaction samples by measuring the distance between samples in the feature space and generating new samples along the line connecting similar transactions. This increases the density of the data and enriches the feature representation, as illustrated in Eq. 2.7.

$$\mathbf{x}_{\text{newfraud}} = \mathbf{x}_i + \lambda \times (\mathbf{x}_j - \mathbf{x}_i) \tag{2.7}$$

However, when the feature boundaries between fraud transactions and legitimate transactions are not clearly distinguishable in the feature space, fraud samples often lie near the boundary of the majority class (legitimate transactions). This presents a challenge in CCFD tasks, as the generated fraud transaction features may overlap with those of legitimate transactions. To address this, Han et al. [26] proposed Borderline-SMOTE, which focuses sampling on the feature space where fraud transactions are more clearly distinguished from legitimate transactions. This method enhances the distinctiveness of fraud transaction features. Nonetheless, the challenge in CCFD remains the overlapping boundary between legitimate and fraud features. Avoiding sampling in these overlapping regions may help mitigate the issue of feature homogenization between different classes, but its effectiveness is limited in environments where the feature space boundaries are indistinct.

GAN-based: Resampling methods like SMOTE [9] often struggle to handle overlapping boundaries between different classes, and ensuring that the generated data closely resembles real-world data remains a significant challenge. Goodfellow et al.[24] proposed a generative adversarial approach to data generation, which aims to generate synthetic data that closely approximates the target real data. Specifically, in the context of CCFD tasks, where distinguishing between fraud and legitimate transactions based on certain features is difficult, GAN-based methods can be particularly effective. The discriminator is trained to distinguish between real fraud transactions and the synthetic samples generated by the generator, thereby forcing the generator to produce samples that closely mimic real fraud transactions in Eq. 2.8.

$$\min_G \max_D \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{2.8}$$

However, this loss function can lead to the issue of gradient vanishing if there is no overlap between the distributions of the generated fraud transactions and the real fraud transactions. To address this, Gulrajani et al.[25] introduced WGAN-GP, an improvement over WGAN-GP [25], by incorporating a gradient penalty term that alleviates the gradient vanishing problem and enhances the stability of training. The gradient penalty ensures that the critic (discriminator) operates under the Lipschitz continuity condition without the limitations imposed by weight clipping [25]. Nonetheless, although WGAN-GP [25] improves the stability of the training process and the quality of the generated samples, the training cost remains high, especially when dealing with high-dimensional datasets. Additionally, the generator may sometimes over-optimize, leading to a lack of diversity in the generated samples, which could result in insufficient variability in the generated fraud transactions, potentially affecting the model's ability to generalize across different types of fraud.

VAE-based: By generating synthetic data similar to fraud transactions, models can enhance their ability to learn from these rare occurrences. However, GANs often suffer from issues such as insufficient variability, and the gradient vanishing or slow convergence when the generated samples and real data distributions do not overlap or overlap very minimally. As a result, unsupervised learning methods like autoencoders (AE) have become increasingly popular in CCFD tasks. These models map transaction features into a latent space, where parameters are updated through gradient descent to learn latent representations of different types of transaction features [72, 20, 48]. Kingma et al. introduced the Variational Autoencoder (VAE) [59], which differs from AE in that instead of mapping to a fixed vector, VAE [59] maps the input into a probabilistic space. This allows VAE [59] to reconstruct the data by generating new samples that are similar to the original data. The diffusion-based generation in VAE [59] not only preserves the original data's characteristics but also enhances the potential representational capacity and diversity of features. As shown in Eq. 2.9, the reconstruction loss constrains the reconstructed features to maintain a primary similarity with the original features, while the KL divergence ensures diversity in the generated samples, preventing the model from focusing excessively on a single region of the feature space while also avoiding excessive dispersion that could lead to distortion.

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}) = -\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] + \text{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \tag{2.9}$$

However, the original distribution's characteristics may lead to excessive dispersion, depending on the task requirements. To address this, Sohn et al. [59] introduced a conditioning mechanism that tailors the distribution according to the task's needs. Specifically in fraud detection tasks, Alazizi et al. [1] applied conditioning constraints alongside supervised model training to better capture fraud-related patterns.

In this study, following the methods of Sohn et al. [59] and Alazizi et al. [1], label information is used as a condition to guide the model during the diffusion process. However, not all labels equally influence the diffusion process. Therefore, a multi-head attention mechanism is introduced to adaptively learn the weights of label information, which are then applied to a new condition to better supervise the model's learning process.

Chapter 3

Project Execution

3.1 The Proposed Approaches

This section begins with a definition of the problem of CCFD. It then details the data preprocessing inputs and the process for using Ma-CVAE to enhance feature diversity. This is followed by a description of the modeling process using TGAT for learning feature representations over time. The method for scoring fraud detection is then discussed. Finally, the process for loss optimization and the methods used for model evaluation are presented.

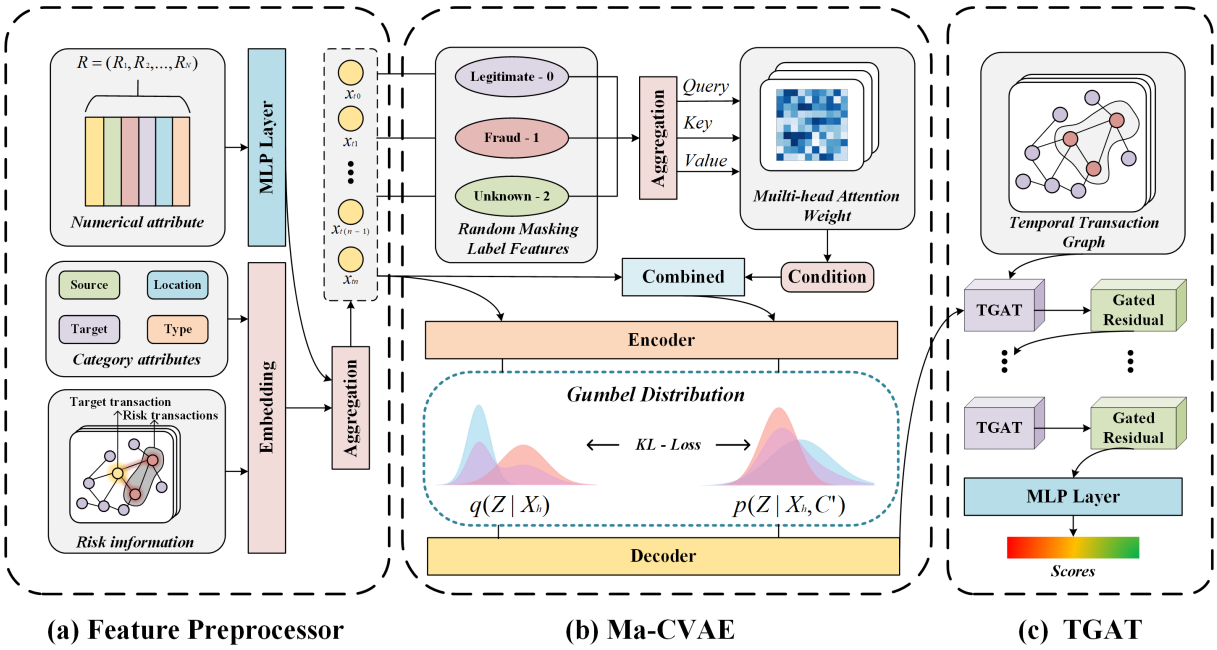


Figure 3.1: The illustration of the overview of the adaptivae semi-supervied method for CCFD. The task encompasses three primary components: the Feature Preprocessor, Ma-CVAE, and TGAT. It starts with the input of Numerical attributes X_{na}, Categorical attributes X_{ca}, and Risk information X_r. The Ma-CVAE then processes these inputs, using three types of labels as conditions to guide the learning of latent feature diversities. Following this, the TGAT[69] is employed to learn the spatiotemporal feature representations. Finally, an MLP layer outputs the scores for CCFD.

3.2 Problem Formalization

Given a series of credit card transactions defined as $\mathbf{R} = \{R_1, R_2, \ldots, R_n\}$, where each transaction R_n is composed of various attributes $R_n = \{\mathbf{S}, \mathbf{L}, \mathbf{Ta}, \mathbf{Ty}\}$. Here, $\mathbf{S}$ represents the source, encoding the origin of the transaction initiator; $\mathbf{L}$ stands for location, encoding the place of the transaction; **Ta**

indicates the target, representing the party initiating the transaction, such as the merchant or other entity receiving the payment; **Ty** denotes the type of transaction, including categories like online shopping or cash withdrawals, each with different fraud risk characteristics. The subset $\mathbf{D} \subseteq \mathbf{R}$ comprises the target transaction events for fraud detection. The objective is to predict the probability of credit card fraud in the target transaction t_{i+1} based on historical transaction records $[t_1, t_i]$.

3.3 Model Architecture

3.3.1 Feature Preprocessing and Attribute Embedding

Following Xiang et al.[69], it was decided not to exclude users with only a few authorized transaction records but to retain all transaction records of users. This decision is based on the practical observation that the number of users without manual review significantly exceeds those who have been reviewed. To capture all possible fraud activities and potential fraud possibilities, retaining all transaction records ensures that no potential anomalies or fraud cases are missed in the dataset.

Therefore, to construct the time-series features of historical transaction data, all transactions in the dataset **R** are first sorted in chronological order based on their timestamps. This process allows for the extraction of transaction features within each time window, including historical transaction records $[t_1, t_i]$ and the subsequent target transaction t_{i+1}, with the label of the target transaction serving as the new label for this time window for model training. Inspired by Fu et al.[22], for each time window, in-depth analyses are performed to calculate the average, total, and standard deviation of transaction amounts, as well as the difference between the current transaction amount and the average amount within the window.

Feature Types	**Description**
Trans amount avg	Average amount of the transactions
Trans amount total	Total amount of the transactions
Trans amount stddev	Standard deviation of the transaction amounts during the past period
Trans amount bias	The difference between the amount of this transaction and the average
Trans count	Total number of the transactions
Trans target count	Number of unique target accounts involved in transactions
Trans location count	Number of unique transaction locations
Trans type count	Number of unique transaction types

Table 3.1: Given a time window T, the data for each transaction and account type is aggregated and further processed to derive eight new features. The table includes descriptions of each feature type for CCFD tasks.

Following Cheng et al.[13], the number of transactions, as well as the number of different targets (merchants), transaction locations, and types of transactions within the time window, are counted. These computed features, combined with the original transaction attributes (source **S**, location **L**, target **Ta**, type **Ty**), form a rich feature set as inputs, denoted as $\mathbf{U} \in \mathbb{R}^{N \times d}$ for the CCFD model. Here, N represents the number of transaction event records, corresponding to the number of transactions, and d represents the dimension of the features.

To enhance feature representation, following Xiang et al. [69], a time window T is set to calculate transaction features within that specific period. The numerical attributes, such as the transaction amount and the number of distinct categories associated with the transaction user, are computed for this time window, resulting in the eight numerical features listed in Table 3.1. These features, when combined with the original numerical attributes of the target transaction, form a new set of numerical attributes $\mathbf{X_{na}}$. This time-window-based numerical feature information better reflects the temporal correlation and potential relationships between transactions, which cannot be captured by the numerical features of a single, isolated transaction.

As shown in Figure 3.1(a), the model for CCFD comprises feature preprocessing and risk embedding, where the Feature Preprocessor constructs three inputs: Numerical attributes $\mathbf{X_{na}}$, Categorical attributes $\mathbf{X_{ca}}$, and Risk information $\mathbf{X_r}$ from $\mathbf{U} \in \mathbb{R}^{N \times d}$.

The numerical attributes are extracted from $\mathbf{U}$ to get $\mathbf{X_{na}} \in \mathbb{R}^{N \times d_{na}}$, aligning with the dimensions of $\mathbf{U}$. The categorical attributes are one-hot encoded to obtain $\mathbf{X_{ca}} \in \mathbb{R}^{N \times d_{ca}}$, where d_{ca} denotes the dimension of categorical attributes and $d_{ca} = d_{na} = d$. The formulas are as follows:

$$\mathbf{X_{ca}} = \text{OneHot}(\mathbf{U}_{\text{categorical}}) \tag{3.1}$$

$$\mathbf{X_{na}} = f_{\text{mlp}}(\mathbf{U}_{\text{numerical}}) \tag{3.2}$$

For the risk information $\mathbf{X_r}$, inspired by Shi et al.[58], a method is proposed that unifies label propagation and feature propagation in a single framework, differing from traditional methods that propagate labels and features separately and ignore the potential synergy between them. This unified approach allows for simultaneous node feature updates and node label propagation. Specifically, the manually annotated labels ('legitimate' and 'fraud') for each transaction are treated as a node feature. For unlabeled nodes, a special label ('unlabeled') is used. These labels are mapped into the feature space through a risk embedding layer denoted as f_e, resulting in risk information $\mathbf{X_r} \in \mathbb{R}^{N \times d_r}$, where $d_r = d$, matching the feature dimensions. The 'unlabeled' nodes will have a zero embedding, providing no risk information.

$$\mathbf{X_r} = f_e(\mathbf{Y_r}) \quad \text{where} \quad \mathbf{Y_r} \in \{\text{'unlabeled'}, \text{'fraud'}, \text{'legitimate'}\} \tag{3.3}$$

Then, the categorical attributes $\mathbf{X_{ca}}$, the numerical attributes $\mathbf{X_{na}}$, and the risk information $\mathbf{X_r}$ are combined to form the new node feature representation $\mathbf{X_h} \in \mathbb{R}^{N \times d}$ as the input to Ma-CVAE in Figure 3.1(b).

$$\begin{aligned} \mathbf{X_h} &= \text{OneHot}(\mathbf{U}_{\text{categorical}}) + f_{\text{mlp}}(\mathbf{U}_{\text{numerical}}) + f_e(\mathbf{Y_r}) \\ &= \mathbf{X_{ca}} + \mathbf{X_{na}} + \mathbf{X_r} \end{aligned} \tag{3.4}$$

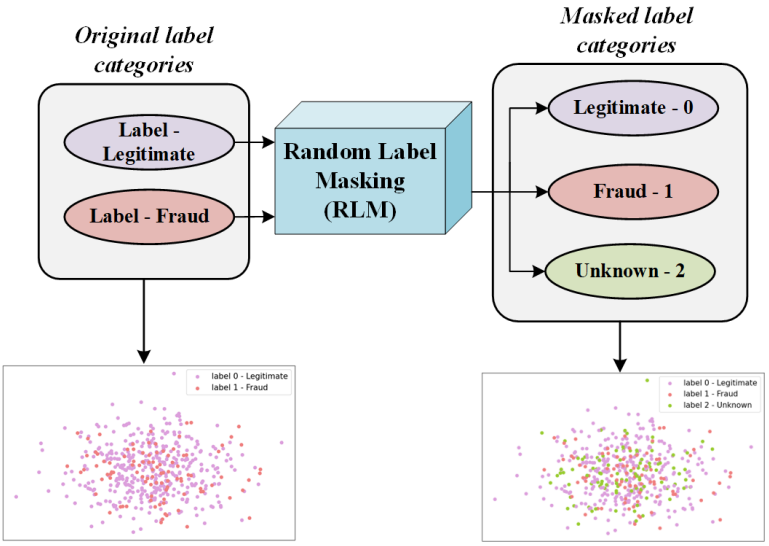


Figure 3.2: The illustration of the random label masking process. Based on the original labels, some of the labels are randomly masked and set as unknown, represented by label 2. As shown in the scatter plot, the green points represent the unknown new label obtained by randomly masking the original purple and red labels. This process is used to construct a semi-supervised learning dataset.

3.3.2 Muiltihead Attention - Conditional Variational Autoencoder

In the dataset, many transaction records are unlabelled. The process of manually labelling these records is labor-intensive and time-consuming. To simulate a realistic scenario where a substantial portion of the data remains unlabelled, as shown in Figure 4.1, a Random Label Masking (RLM) model is employed. This model randomly masks a portion of the labels from the original dataset, converting them to a value of 2 to indicate 'unknown', as shown in the scatter plot in Figure 3.2, where purple points represent legitimate labels (0) and red points represent fraud labels (1). The Random Label Masking module randomly masks some labels to 2, indicating 'unknown', thus constructing semi-supervised data. This approach helps avoid overfitting in the input label information of self-loops [58] and mitigates overfitting

issues in GNN-based models [23, 52]. The RLM-processed labels are then used as inputs for the Ma-CVAE model, as depicted in Figure 3.1(b), denoted as $X_l \in \mathbb{R}^N$.

For X_l, similar to $\mathbf{X_r}$, the label embedding function $f^{'}$ is first used to map it to the dimension of spatial features, resulting in $X_l^{'} \in \mathbb{R}^{N \times d}$, which is the combined feature representation of three categories of labels. The formula is as follows:

$$\mathbf{X_l^{'}} = f^{'}(X_l) \tag{3.5}$$

However, not every transaction's label feature positively impacts model training. Some transactions, although labeled as legitimate, have similar label features to fraud labels that do not distinctly differentiate between the two categories. This lack of distinction might not contribute positively to model training and could, in fact, mislead the model learning process. Therefore, inspired by Ashish et al. [65], a Multi-Head Attention mechanism is used to enhance the feature representation from multiple perspectives. As shown in Figure 3.3, this approach aims to better understand the significance of different label features for the model, focusing on those with clear distinguishing characteristics. The formulas are shown below:

$$\mathbf{X_l^{'}} = \text{unsqueeze}(\mathbf{X_l}) \tag{3.6}$$

$$\mathbf{Q_l} = \mathbf{W_q}\mathbf{X_l^{'}} = \begin{bmatrix} \mathbf{Q_1} \\ \mathbf{Q_2} \\ \mathbf{Q_3} \\ \vdots \\ \mathbf{Q_h} \end{bmatrix}, \quad \mathbf{K_l} = \mathbf{W_k}\mathbf{X_l^{'}} = \begin{bmatrix} \mathbf{K_1} \\ \mathbf{K_2} \\ \mathbf{K_3} \\ \vdots \\ \mathbf{K_h} \end{bmatrix}, \quad \mathbf{V_l} = \mathbf{W_v}\mathbf{X_l^{'}} = \begin{bmatrix} \mathbf{V_1} \\ \mathbf{V_2} \\ \mathbf{V_3} \\ \vdots \\ \mathbf{V_h} \end{bmatrix} \tag{3.7}$$

In Eq. 3.6, $\mathbf{X_l^{'}}$ is unsqueezed to add one dimension, transforming it into $\mathbb{R}^{n \times 1 \times d}$, and then passed through respective single-layer fully connected layers, multiplied by the weights $\mathbf{W_q} \in \mathbb{R}^{d \times k}$, $\mathbf{W_k} \in \mathbb{R}^{d \times k}$, $\mathbf{W_v} \in \mathbb{R}^{d \times d_v}$. Subsequently, the scaled dot-product attention scores are computed, assigning higher scores to labels with distinctive features, focusing on and mapping to the interval [0,1] to obtain the attention matrix as shown in Eq. 3.8. Then, this attention matrix is multiplied by $\mathbf{V_l}$ to obtain label features with focused processing. Simultaneously, the output for each head represents the relationship between label features viewed from different perspectives, thereby avoiding the attention bias towards a small part of important features [18].

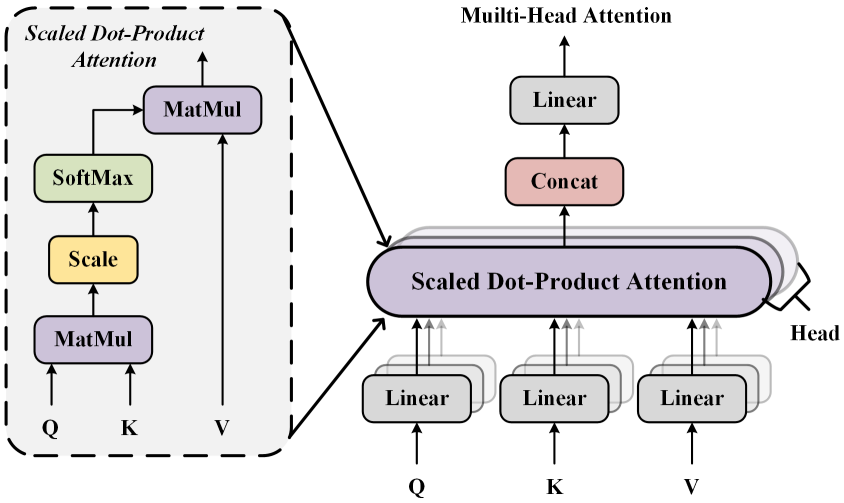


Figure 3.3: The illustration of the Multi-Head Attention mechanism following the self-attention mechanism [65]. In this mechanism, the input label features are transformed into queries, keys, and values. The similarity between the queries and all keys is calculated, resulting in a set of weights through a Softmax layer. These weights are then used to compute a weighted sum of the corresponding values, yielding the final value in the attention mechanism.

$$\mathbf{A} = \text{Softmax}(\frac{\mathbf{Q_l}\mathbf{K_l}^T}{\sqrt{k}}) \tag{3.8}$$

$$\mathbf{X_h} = \mathbf{A}\mathbf{V_l} \tag{3.9}$$

where $\mathbf{A} \in \mathbb{R}^{n\times h\times k}$ represents the dot-product attention matrix for each head. The attention matrixs are normalized using Softmax to obtain the attention weights attn_weights $\in \mathbb{R}^{n\times h\times k}$. Then, the attention output attn_output $\in \mathbb{R}^{n\times h\times d_v}$ is obtained by taking the weighted sum of $\mathbf{V_l}$.

$$\mathbf{X_a} = \mathbf{W_h}\text{Concat}(\text{x}_1, \text{x}_2, \ldots, \text{x}_{\text{head}}) \tag{3.10}$$

$$\mathbf{X_a}' = \text{squeeze}(\mathbf{X_a}) \tag{3.11}$$

$$\mathbf{C} = \text{LN}(\mathbf{X_a}') + \mathbf{X_l}' \tag{3.12}$$

where $\mathbf{W_O} \in \mathbb{R}^{hd_v\times d}$ is the linear transformation matrix, and Concat denotes the concatenation operation. The resulting matrix is $\mathbf{X_a} \in \mathbb{R}^{n\times 1\times d}$. Then, by applying the squeeze operation, the second feature dimension is removed, resulting in $\mathbf{X_a}' \in \mathbb{R}^{n\times d}$, as shown in Eq. 3.11. The residual of $\mathbf{X_a}'$ and the original multi-head attention input $\mathbf{X_l}'$ are then combined and layer normalized to produce $\mathbf{C} \in \mathbb{R}^{n\times d}$, which is used as a condition for the subsequent Gumbel distribution sampling in the latent space[33].

Inspired by Sohn et al. [59], most supervised learning tasks face the challenge of learning complex representations, which can easily lead to representation bias. Similarly, in the CCFD task, there is a challenge in learning complex and diverse fraud features. To address this, as shown in Figure 3.4, features are sampled from a latent space under the guidance of conditional prior information, such as label information, which is treated as a condition on input observations. This approach enhances the model's ability to learn diverse and complex structures in the CCFD task.

In Figure 3.4, approach 1 commonly involves sampling from a Gaussian distribution, using μ and σ as the mean and standard deviation, respectively, as described in Eq. 3.13. This method enables the model to capture the underlying patterns in the data by considering the conditional distribution, thereby improving the representation of features that are critical for detecting fraudulent activities.

$$\mathbf{Z}_{\text{gaussian}} = \mu + \sigma \cdot \epsilon \tag{3.13}$$

However, the traditional Gaussian distribution obtained through sampling cannot adequately capture certain discrete features, such as categorical features in CCFD, and may easily lead to gradient propagation issues, especially due to the mismatch between the Gaussian distribution and the distribution of discrete data, resulting in a decline in training and generation quality. Therefore, following Jang et al. [33], a discrete sampling process is used, approximated as a differentiable Gumbel distribution sampling process in the following equation:

$$\mathbf{Z}_{\text{logits}} = \text{Encoder}(\text{Concat}(\mathbf{X_h}, \mathbf{C}')) \tag{3.14}$$

where the input features $\mathbf{X_h}$ and the condition $\mathbf{C}'$, weighted by Attention, are first concatenated together. Then, they are passed through the encoder layer composed of fully connected layers to obtain $\mathbf{Z}_{\text{logits}} \in \mathbb{R}^{n\times d}$.

In Figure 3.4, approach 2, which follows Jang et al. [33], utilizes the Gumbel distribution as described in Eq. 3.15. In the code implementation, this is achieved by generating uniformly distributed random numbers $\mathbf{u}$ over the interval $(0, 1)$, as shown in Eq. 3.16. Each logit is then perturbed by adding the corresponding Gumbel noise to facilitate the sampling process. This approach allows for a differentiable approximation of categorical variables, enabling the backpropagation of gradients through the sampling operation.

$$\text{Gumbel-g}(\mu, \beta) = \frac{1}{\beta}\exp\left(-\frac{x-\mu}{\beta} + \exp\left(-\frac{x-\mu}{\beta}\right)\right) \tag{3.15}$$

$$\mathbf{g} = -\log(-\log(\mathbf{u})) \tag{3.16}$$

Using the Gumbel-Max trick for reparameterization, in Eq. 3.17, the one-hot encoded variable $\mathbf{Z}$ is obtained as a discrete latent variable in the latent space using the following equation:

$$\mathbf{Z} = \text{Onehot}(\arg\max_i[\mathbf{g}_i + \log \mathbf{Z}_{\text{logits},i}]) \tag{3.17}$$

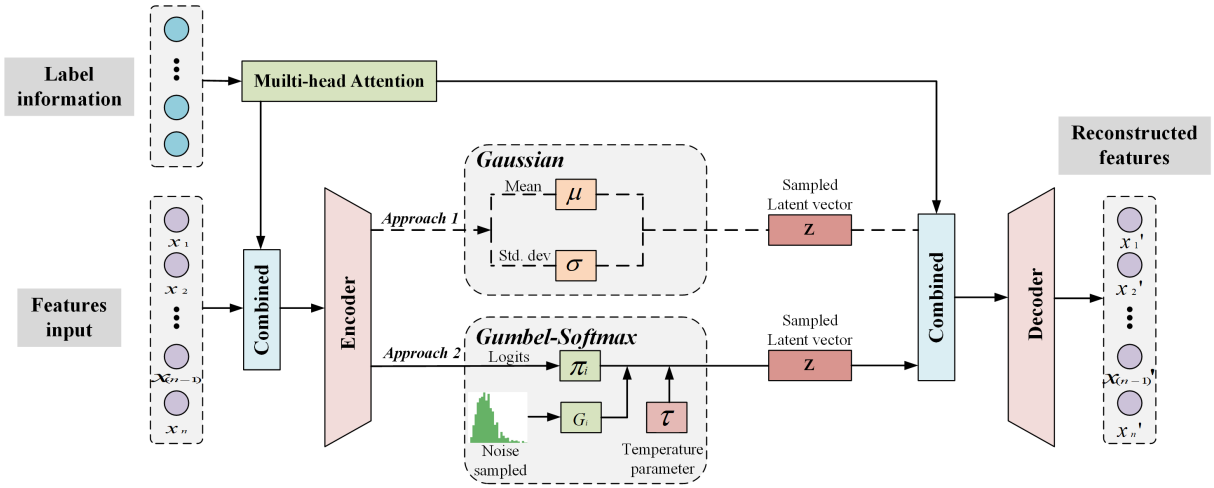


Figure 3.4: An illustration of the Ma-CVAE model, demonstrating the two approaches used to map data through different distributions: Gaussian distribution and Gumbel-Softmax distribution [33]. These methods are used to learn the latent representations of the probabilistic distribution of features while generating reconstructed features that are similar to the original data under the supervision of label information(condition), the label information, which serves as the condition in this context.

where i represents different categories of features. However, the arg max function used in Eq. 3.17 is not differentiable. Inspired by Jang et al. [33], a softmax function is used to approximate and simplify this process as shown in Eq. 3.18, resulting in $\mathbf{Z} \in \mathbb{R}^{n \times d}$.

$$\mathbf{Z} = \text{Softmax}\left(\frac{\mathbf{Z}_{\text{logits}} + \mathbf{g}}{\tau}\right) \tag{3.18}$$

In Eq. 3.18, τ serves as a hyperparameter for feature distribution. As shown in Figure 3.5, a too small value of τ will make the distribution sharper, causing the model to overfit to specific features while neglecting others, which leads to overshooting during training. Therefore, τ is set to a relatively large value ($\tau \geq 10$) to achieve a smoother distribution, promoting a more uniform learning process across all features.

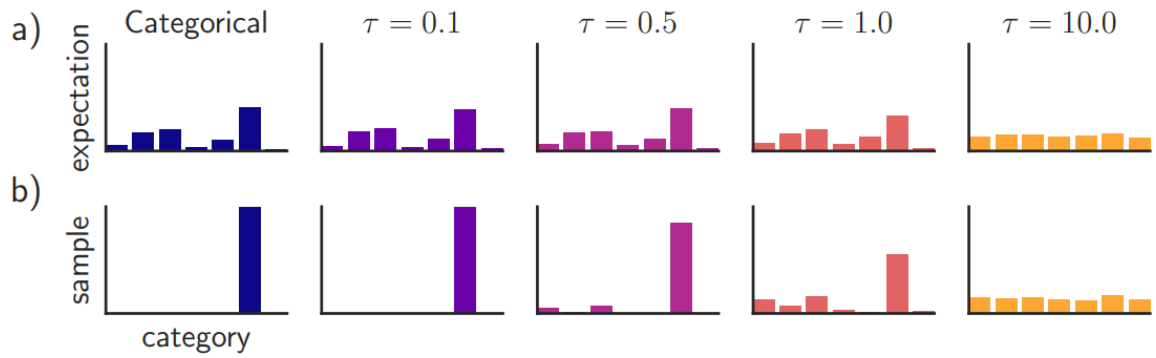


Figure 3.5: Cited from Jang et al. [33], this figure illustrates the sample distribution results for categorical features under different values of τ. As shown in the figure, when $\tau = 10$, the distribution approaches a uniform distribution, while when τ is very small, such as 0.1, it focuses on one specific category feature. In the CCFD task, specifically in the FFSD dataset, there are categorical and numerical features, which are combined for feature learning. However, the Gaussian distribution sampling process cannot directly achieve this, potentially leading to gradient issues during backpropagation. Therefore, Gumbel distribution sampling is used to balance the sampling process of discrete and continuous features, and a relatively large value of τ is chosen to make the distribution more uniform.

Compared to the traditional VAE proposed by Kingma and Welling [36], the Ma-CVAE introduced

in Eq. 3.14 incorporates a condition as an additional variable for guided learning in specific scenarios. This enhancement enables the model to better capture the latent distribution of data points, aligning with the unique characteristics of each transaction.

By expanding the model's exploration within the latent space and reducing its reliance on dominant categories in the training data—both in terms of labels and features—this approach enhances diversity among the three categories of label features, effectively mitigating bias caused by label and feature imbalances.

Then, the resampled result $\mathbf{Z}$ is combined with the condition $\mathbf{C}'$ and passed through a decoder composed of fully connected layers to obtain $\mathbf{X}'_{\mathrm{h}} \in \mathbb{R}^{n \times d}$.

$$\mathbf{X}'_{\mathrm{h}} = \mathrm{Decoder}(\mathbf{Z} \parallel \mathbf{C}') \tag{3.19}$$

Furthermore, combining $\mathbf{X}'_{\mathrm{h}}$ with the input $\mathbf{X}_{\mathrm{h}}$, the output of Ma-CVAE $\mathbf{X}_{\mathrm{cvae}}, \in \mathbb{R}^{n \times d}$ is obtained:

$$\mathbf{X}_{\mathrm{cvae}} = \mathbf{X}'_{\mathrm{h}} + \mathbf{X}_{\mathrm{h}} \tag{3.20}$$

This process generates a distribution reconstruction, which includes comparing the KL divergence between the prior distribution and the posterior distribution, as detailed in Section 3.4.2.

3.3.3 Gated Temporal Attention Network

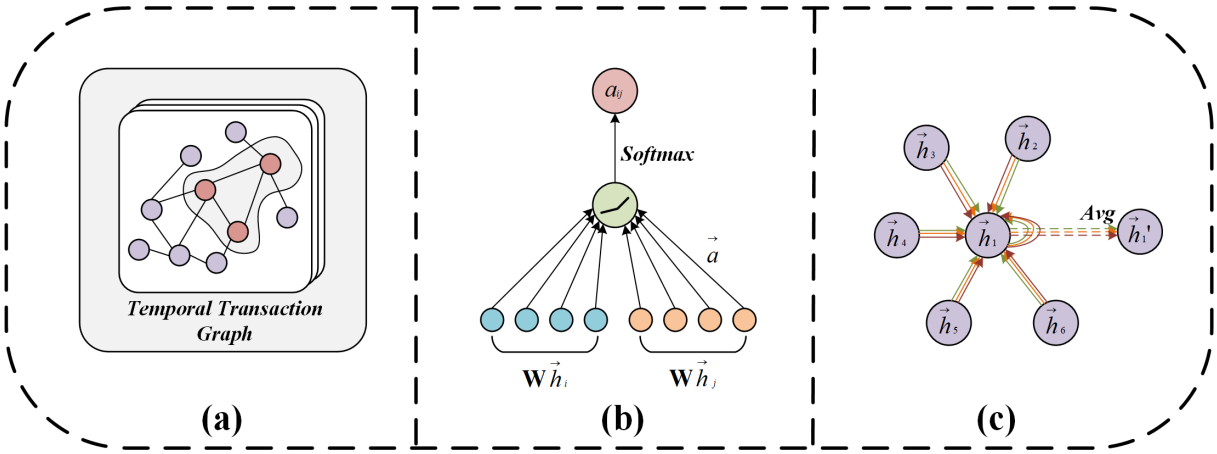


Figure 3.6: This illustration demonstrates the working principle of the TGAT model, which applies a Temporal Transaction Graph based on the Graph Attention Networks (GAT) [66]. The figure is composed of three subfigures: (a) shows the Temporal Transaction Graph obtained through data processing; (b) illustrates how each node in the Temporal Transaction Graph calculates the Attention by applying learned weight parameters to the features of both its neighbors and itself, followed by a softmax activation function; (c) depicts the multi-head attention mechanism, where each color represents a different attention head. Compared to a single attention mechanism, the multi-head attention mechanism is better at learning the latent representations of features.

After the Ma-CVAE generates the latent feature representations, temporal information features can be represented as $\mathbf{X}_{\mathrm{temp}} = \{x_{t_0}, x_{t_1}, \ldots, x_{t_n}\}$, where $x_{t_i} = x_{\mathrm{n}}^{t_i} + x_{\mathrm{c}}^{t_i}$. Here, $x_{\mathrm{n}}^{t_i}$ and $x_{\mathrm{c}}^{t_i}$ denote the numerical and categorical features that vary over time, respectively. Setting $\mathbf{h} = \mathbf{X}_{\mathrm{temp}}$, the TGAT model, integrated with a two-layer MLP as proposed by Xiang et al. [69], is employed for risk prediction by learning the temporal features $\mathbf{X}_{\mathrm{temp}}$.

Traditional GCN [38] and GNN [54] models focus on aggregating graph information through static topological structures.

The standard GNN model aggregates information using fixed coefficients for all connected neighbors, meaning that each neighbor's influence is weighted equally or consistently:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \mathbf{W}^{(l)} \mathbf{h}_j^{(l)}\right) \tag{3.21}$$

However, this can lead to overemphasis on noisy nodes or underrepresentation of important features, causing issues such as information overload or dilution.

In terms of the aggregation processGCNs improve upon GNNs by introducing a degree matrix that normalizes the aggregation process, similar to convolutional neural networks, thus ensuring a balanced contribution from nodes with different degrees:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \frac{1}{c_{ij}}\mathbf{W}^{(l)}\mathbf{h}_j^{(l)}\right) \tag{3.22}$$

In Eq. 3.22, $\mathcal{N}(i)$ represents the set of first-order neighbors of node i (including v_i itself due to self-loops), and $c_{ij} = \sqrt{|\mathcal{N}(i)|}\cdot\sqrt{|\mathcal{N}(j)|}$ is a normalization constant based on the graph's structure. The function σ is an activation function, while $W^{(l)}$ is the shared weight matrix used for transforming node features in layer l.

While GCNs address the issue of unbalanced neighbor contributions, they are primarily designed for static graphs, which limits their applicability to dynamic graphs, such as Temporal Graphs. In Temporal Graphs, node features change over time, as shown in Figure 3.6 (a).

In contrastThe TGAT model [69], based on GAT, is particularly effective for such graphs. Unlike GCNs that rely on Laplacian transformations, TGAT utilizes can dynamically assign weights to neighboring node features based on their relevance, as illustrated in Figure 3.6 (b). Each single-head operation in TGAT can be expressed as follows.

In Eq.3.23, a learnable parameter $\mathbf{W}^{(l)} \in \mathbb{R}^{d\times d}$ is used to obtain the hidden layer features through a linear transformation.

$$\mathbf{z}_{ij}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}_i^{(l)} \tag{3.23}$$

Then, the attention score $e_{ij}^{(l)}$ between the target node i and its neighbor node j is calculated. This attention score quantifies the weight of information flow from node j to node i and represents the unnormalized attention score for the edge (i, j) in layer l. Specifically, the features of the target node and its neighbor, $\mathbf{z}_i^{(l)}$ and $\mathbf{z}_j^{(l)}$, are concatenated. Then, this concatenated feature vector is multiplied by a learnable weight vector $\mathbf{a}^{(l)T}$, and the result is passed through the LeakyReLU activation function to obtain the attention score:

$$e_{ij}^{(l)} = \text{LeakyReLU}\left(\mathbf{a}^{(l)T}\cdot\left[\mathbf{z}_i^{(l)} \parallel \mathbf{z}_j^{(l)}\right]\right) \tag{3.24}$$

The attention score $e_{ij}^{(l)}$ is then normalized to map it to the range $[0, 1]$, ensuring that the sum of attention scores for each target node across its different neighbors equals 1:

$$\alpha_{ij}^{(l)} = \frac{\exp(e_{ij}^{(l)})}{\sum_{k\in\mathcal{N}(i)}\exp(e_{ik}^{(l)})} \tag{3.25}$$

After obtaining the attention scores, the features of all neighboring nodes of the target node are aggregated together, weighted by their respective attention scores. This aggregated feature vector is then passed through the activation function σ to provide a non-linear transformation, generating the feature representation for the next layer:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(l)}\mathbf{z}_j^{(l)}\right) \tag{3.26}$$

To more comprehensively learn the latent representations of features, a multi-head mechanism is employed. Similar to the discussion in Figure 3.3, a single-head attention mechanism tends to focus on a specific type of relationship, which can negatively impact the model when facing noisy relationships. The feature representation $\mathbf{h}_i^{(l+1)}$ for node i at layer $l+1$ is calculated as follows:

$$\mathbf{h}_i^{(l+1)} = \|_{k=1}^{K}\ \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{k}\mathbf{W}^{k}\mathbf{h}_j^{(l)}\right) \tag{3.27}$$

3.3.4 Gated Residual

To maintain a balance between preserving original information and introducing new information, and to prevent either from excessively influencing the model's training, which could lead to increased training error as the network depth increases, an Attribute-driven Gated Residual approach, as proposed by Xiang et al.[69], is employed. This approach provides a shorter backpropagation path, allowing gradients to flow more easily to the shallower layers, effectively mitigating the problem of gradient vanishing or explosion.

A learnable parameter $\mathbf{W}_{\text{skip}}$ is used to perform a linear transformation on the feature $\mathbf{h}_i^{(l)}$ of node i at layer l, yielding the skip feature $\mathbf{h}_{\text{skip}}$. This allows the feature from the previous layer to be introduced into the current layer:

$$\mathbf{h}_{\text{skip}} = \mathbf{W}_{\text{skip}}\mathbf{h}_i^{(l)} \tag{3.28}$$

Subsequently, a gating value $\mathbf{g}_i$ is introduced to determine the fusion ratio between the skip feature $\mathbf{h}_{\text{skip}}$ and the new feature $\mathbf{h}_i^{(l+1)}$. Specifically, the skip feature, the new feature, and their difference are first concatenated, then passed through a learnable weight matrix $\mathbf{W}_{\text{gate}}$ for a linear transformation, and finally processed by an activation function σ to obtain the gating value $\mathbf{g}_i$. The gating value ranges between [0, 1] and is used to control the degree of mixing between the original information and the new information in the final output. The formula is as follows:

$$\mathbf{g}_i = \sigma\left(\mathbf{W}_{\text{gate}}\left[\mathbf{h}_{\text{skip}} \parallel \mathbf{h}_i^{(l+1)} \parallel \left(\mathbf{h}_{\text{skip}} - \mathbf{h}_i^{(l+1)}\right)\right]\right) \tag{3.29}$$

where $\parallel$ denotes the concatenation of the input vectors.

The output feature representation $\mathbf{h}_i^{(l+1)}$ is then computed as a weighted sum of the skip feature and the newly aggregated feature, controlled by the gating value:

$$\mathbf{h}_i^{(l+1)} = \mathbf{g}_i \odot \mathbf{h}_{\text{skip}} + (1 - \mathbf{g}_i) \odot \mathbf{h}_i^{(l+1)} \tag{3.30}$$

where $\odot$ denotes the element-wise multiplication. By using this Gated Residual, as opposed to a simple concatenated residual, the model can adaptively balance the original features and the transformed features. This adaptive balancing allows for better control over the mixing degree of the original information and the new information in the output.

Then, to accelerate the convergence process, layer normalization and an activation function are applied to further process the output features

$$\mathbf{X}_{\text{out}} = \sigma(\text{LayerNorm}(\mathbf{h}_{\text{out}})) \tag{3.31}$$

where $\mathbf{X}_{\text{out}}$ is the final output representation of the TGAT model, σ denotes the activation function, and LayerNorm represents the layer normalization operation applied to the input feature $\mathbf{h}_{\text{out}}$.

3.4 Loss Optimization

3.4.1 Fraud Risk Prediction

After obtaining the output $\mathbf{X}_{\text{out}}$ from TGAT [69], it is used in a multi-layer MLP to calculate the probability of a fraud label in the CCFD task. The activation function is used to map the probability to the range [0,1].

$$\hat{\mathbf{r}} = \sigma(\alpha(\mathbf{X}_{\text{out}}\mathbf{W}_1 + \mathbf{c}_1)\mathbf{W}_2 + \mathbf{c}_2) \tag{3.32}$$

where $\hat{\mathbf{r}} \in \mathbb{R}^{M\times 1}$ denotes the high-risk fraud probability results of each transaction in the CCFD task, and $\mathbf{W}$ and $\mathbf{c}$ denote the learnable parameters of the fully connected layer. Subsequently, a cross-entropy is used to measure the quality of the model predictions, resulting in the loss function for each training iteration, which is used for backpropagation. The loss function $\mathcal{L}_1$ is calculated as shown below:

$$\mathcal{L}_1 = -\frac{1}{M}\sum_{j=0}^{M}\left[z_j \cdot \log q(\hat{r}_j|\mathbf{Y},\mathbf{B}) + (1 - z_j) \cdot \log(1 - q(\hat{r}_j|\mathbf{Y},\mathbf{B}))\right], \tag{3.33}$$

where $\mathbf{B}$ represents the prior information input to the model, specifically including transaction type features, numerical features, and risk labels of historical transactions. z denotes the target label of transactions. $\mathbf{M}$ represents the total number of transactions, and j denotes each individual transaction instance.

3.4.2 Reconstruction Loss and KL Loss

In the field of computer vision, to prevent excessive dispersion in VAE-based [34, 28, 74] models from causing reconstruction distortion and making the reconstructed images significantly different from the original ones, thus losing their discriminative value, Kingma et al. [36] introduced a method to mitigate this issue. Similarly, during the process of feature enhancement, to prevent excessive dispersion of features that could lead to learning incorrect features and negatively impact model training, a reconstruction loss is introduced. This loss computes the mean squared error (MSE) between the input features and output features, serving to constrain the excessive dispersion during backpropagation. The formula is as follows:

$$\mathcal{L}_{\text{re}} = \text{MSE}(\mathbf{X}'_{\text{h}}, \mathbf{X}_{\text{h}}) = \|\mathbf{X}'_{\text{h}} - \mathbf{X}_{\text{h}}\|^2 \tag{3.34}$$

where $\mathbf{X}_{\text{h}}$ and $\mathbf{X}'_{\text{h}}$ represent the input and output of the CVAE component, respectively.

As described in Section 3.3.2, after obtaining the resampled spatial features, to prevent the features from being either concentrated or unbounded in the space, which would result in a non-standard Gumbel-g(μ, β) distribution, a KL divergence is used to measure the loss between the approximate posterior distribution $q(\mathbf{z}|\mathbf{x}, \mathbf{c})$ and the predefined prior distribution $p(\mathbf{z}|\mathbf{c})$. This process helps generate more reasonable spatial features. The formula is as follows:

$$\begin{aligned} \mathcal{L}_{\text{KL}} &= D_{\text{KL}}\left(q(\mathbf{Z}|\mathbf{X_h}, \mathbf{C}')\|p(\mathbf{Z}|\mathbf{C}')\right) \\ &= \mathbb{E}_{q(\mathbf{Z}|\mathbf{X_h}, \mathbf{C}')}\left[\log q(\mathbf{Z}|\mathbf{X_h}, \mathbf{C}') - \log p(\mathbf{Z}|\mathbf{C}')\right] \end{aligned} \tag{3.35}$$

where $\mathbf{X_h}$ represents the feature input to the Ma-CVAE model, and $\mathbf{C}'$ denotes the conditional information used for sampling.

Then, the reconstruction loss and KL divergence loss are combined with a certain weighting factor, α, to form the total loss.

$$\begin{aligned} \mathcal{L}_2 &= \mathcal{L}_{\text{re}} + \alpha\mathcal{L}_{\text{KL}} \\ &= \text{MSE}(\mathbf{X}'_{\text{h}}, \mathbf{X}_{\text{h}}) + \alpha D_{\text{KL}}\left(q(\mathbf{z}|\mathbf{x}, \mathbf{c})\|p(\mathbf{z}|\mathbf{c})\right) \end{aligned} \tag{3.36}$$

3.4.3 Total Loss

Finally, the model's fraud risk prediction loss $\mathcal{L}_1$ and the combined result of the reconstruction loss and KL divergence from the Ma-CVAE $\mathcal{L}_2$ are weighted by a factor β and summed together. This allows the model to update its parameters through backpropagation based on the total loss. The formula is as follows:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_1 + \beta\mathcal{L}_2 = \mathcal{L}_1 + \beta\left(\mathcal{L}_{\text{re}} + \alpha\mathcal{L}_{\text{KL}}\right) \tag{3.37}$$

3.5 Prevent Label Information Leakage

In previous work on the CCFD task, risk information was used as a supervisory signal during training to update the model parameters through backpropagation, while the risk information was removed during testing to avoid information leakage. However, this discrepancy between the information input during training and testing can lead to overfitting, resulting in poor performance on the test set. To address this issue, Xiang et al.[69] proposed a masking approach to prevent risk information leakage. Specifically, the labels of target nodes are masked during both training and testing, forcing the model to learn risk information from the neighboring transactions of each node.

To further prevent information leakage through direct connections with 1-hop and 2-hop neighbors, the risk information of these neighbors is set to zero, corresponding to the 'unknown' category in the dataset, while the other information remains unchanged. The target labels $\hat{\mathbf{Y}}$ are masked to prevent information leakage, and combined with risk information, they are used as prior information for the model's supervised learning to predict $\hat{\mathbf{Y}}$ with given $\mathbf{X}_{\text{h}}$, $\tilde{\mathbf{Y}}$, and $\mathbf{B}$:

$$\mathcal{L}_1 = -\frac{1}{|U|}\sum_{i=0}^{|U|}\left[z_j \cdot \log p(\hat{y}_i \mid \mathbf{X}_{\text{h}}, \tilde{\mathbf{Y}}, \mathbf{B}) + (1 - y_i) \cdot \log(1 - p(\hat{y}_i \mid \mathbf{X}_{\text{h}}, \tilde{\mathbf{Y}}, \mathbf{B}))\right], \tag{3.38}$$

where $|U|$ represents the number of target nodes that need to be masked. By using the risk information of the model's neighboring nodes as categorical features and the target nodes' features, this approach helps prevent ground truth information leakage caused by self-loops [58].

3.6 Model Evaluation Method

In the context of CCFD, which is a binary classification task after excluding transactions labeled as 'unknown'.To achieve a comprehensive evaluation, three key metrics will be utilized: AUC, F1, and Average Precision (AP). These metrics will be applied to the FSSD [69], Amazon [34], and YelpChi [49] datasets to quantify the results, as shown in Tables 4.2, 4.3, and 4.4.

Area Under the Curve (AUC) is a classification metric originating from the medical field, specifically referring to the area under the Receiver Operating Characteristic curve. The ROC curve is plotted on a two-dimensional plane, where the x-axis represents the false positive rate (FPR) and the y-axis represents the true positive rate (TPR). For a given classifier, a point on the ROC plane corresponds to a specific TPR and FPR pair based on its performance on the test dataset. The formula is as follows:

$$\text{AUC} = \int_0^1 \text{TPR}(x)\, d\text{FPR}(x) \tag{3.39}$$

By adjusting the classifier's threshold, a curve passing through the points (0, 0) and (1, 1) can be generated, known as the ROC curve. Typically, this curve should lie above the line connecting (0, 0) and (1, 1), as this line represents a random classifier. The AUC quantifies the area under the ROC curve, summarizing the classifier's performance. A value closer to 1 indicates stronger classification ability, while an AUC of 0.5 suggests that the model's performance is equivalent to random guessing, indicating no effective classification capability.

Compared to Accuracy, which may not effectively reflect a model's ability to classify minority classes in imbalanced datasets like FFSD[69], the F1 Score offers a better evaluation metric. In cases where the majority of samples belong to the negative class, Accuracy can be misleading. The F1 Score, as the harmonic mean of Precision and Recall, addresses this challenge effectively. Recall indicates the model's ability to identify positive samples, with higher values suggesting better detection of positive instances. Precision reflects the model's capability to distinguish negative samples, where higher Precision means stronger discrimination of negatives. The formula is as follows:

$$\text{F1 Score} = \frac{2 \times \text{tp}}{2 \times \text{tp} + \text{fp} + \text{fn}} \tag{3.40}$$

The F1 Score balances both Precision and Recall, providing a comprehensive measure of model robustness, particularly in imbalanced scenarios. The higher the F1 Score, the more reliable the classification model is.

Another metric that balances Precision and Recall is the AP. AP is derived from the area under the Precision-Recall curve, where Precision is plotted on the y-axis and Recall on the x-axis. Since the data is discrete, the PR curve is approximated by summing the values at n sampling points and then averaging them. The formula is given by:

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n \tag{3.41}$$

P_n represents the Precision at the nth threshold, and R_n denotes the Recall at the same threshold. AP is particularly useful for evaluating a model's performance on imbalanced datasets by averaging its classification ability across various thresholds. A higher AP indicates better overall performance.

Chapter 4

Critical Evaluation

In this section, the details of the FFSD, Amazon, and YelpChi datasets are first described. Then, the specific experimental parameters, including the number of epochs, learning rate, etc., are presented. Following this, the method for comparing model performance is introduced, and based on this method, the quantitative and qualitative results of the proposed model are compared with other models in fraud detection and anomaly detection. This comparison demonstrates that the proposed method surpasses the current SOTA models.

4.1 Datasets

To thoroughly evaluate the performance of the proposed model and compare it with other baseline models, a semi-supervised financial transaction dataset, the Financial Fraud Semi-supervised Dataset [69], is utilized. Due to the limited availability of open-source semi-supervised financial datasets suitable for GNN-based models, two additional anomaly detection datasets are employed: the Amazon review dataset [34] and the YelpChi review dataset [49].

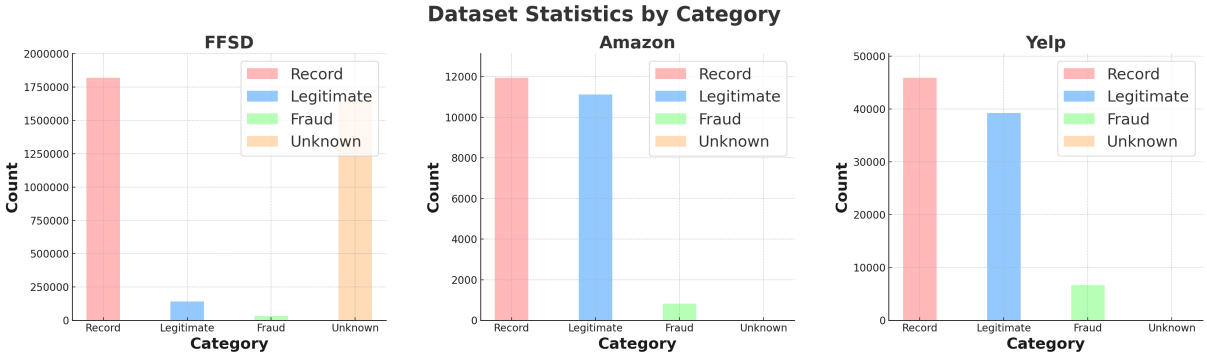


Figure 4.1: The illustration of the FFSD, Amazon, and Yelp datasets, where red represents records, blue represents legitimate records, green represents fraudulent records, and orange represents unlabelled records. These different categories are manually labeled. Due to the significantly larger number of records in the FFSD dataset compared to Amazon and Yelp, the cost of manual labeling increases substantially, resulting in a large number of unlabelled records.

4.1.1 Financial Fraud Semisupervised Dataset

Financial Fraud Semisupervised Dataset is a financial dataset. Due to the lack of publicly available datasets suitable for the semi-supervised CCFD task, Xiang et al. [69] and their collaborators constructed their own dataset, labeling a portion of the fraud data. The resulting Financial Fraud Semi-supervised Dataset (FFSD), as illustrated in Figure 4.1, contains a total of 1,820,840 transaction records, which serve as the nodes in the model, and 31,619,440 transaction connections as edges. Given the high cost of manual labeling, 1,645,121 records, accounting for 90.35% of the dataset, remain unlabeled. The dataset also includes 141,861 records labeled as 0 (Legitimate), representing 7.79% of the total, and 33,858 records

labeled as 1 (Fraud), with an anomaly rate of 1.86% of the total. All labels for legitimate and fraudulent transactions have been verified by both users and experts.

4.1.2 Amazon Dataset

The Amazon dataset was constructed by McAuley et al. [34] in 2013 with the purpose of identifying fraudulent reviews in the Musical Instrument category on Amazon. Three types of relationships are considered: U-P-U, where users have reviewed at least one common product; U-S-U, indicating users who have assigned identical star ratings within the same week; and U-V-U, representing users whose review patterns fall within the top 5% in terms of similarity. The dataset contains a total of 11,948 review records, serving as nodes, and 8,808,728 review connections as edges. Among these, 11,127 are labeled as legitimate reviews, accounting for 93.13% of the dataset, while 821 are labeled as fraudulent, with no unknown labels, resulting in an anomaly detection rate of 6.87%.

4.1.3 YelpChi Dataset

In 2015, Rayana et al. [49] introduced the YelpChi dataset to facilitate the detection of fraudulent reviews in the hotel and restaurant categories on Yelp. The graph constructed from this dataset includes three types of edges: R-U-R, which connects reviews posted by the same user; R-S-R, which links reviews of the same product that share an identical star rating; and R-T-R, which associates reviews of the same product posted within the same month. The dataset comprises a total of 45,954 review records, serving as nodes, with 32 features each, and 3,846,979 review connections as edges. Among these, 39,277 are labeled as legitimate reviews, representing 85.47% of the dataset, while 6,677 are labeled as fraudulent, with no unknown labels. The dataset has an anomaly rate of 14.53%.

4.2 Experiment Settings

Hyperparameter	Description	Value
B	Batch Size	64
H	Hidden Dimension	256
η	Learning Rate	0.002
λ	Weight Decay	1×10^{-4}
L	Number of Layers	4
D_1, D_2	Dropout Rates	0.2, 0.1
ES	Early Stopping	10
K	Number of Folds	5
S	Seed	2024
E	Max Epochs	15
T_s	Test Size	0.6
H_n	Neighbor Attention Heads	Yelp: 4, Amazon: 5, FFSD: 9

Table 4.1: Hyperparameter Settings in the Experiments

To efficiently conduct the experiments, the A100 GPU on the Google Colab cloud platform was rented and utilized. Table 4.1 presents the parameter settings for a series of experiments. A batch size of 64 was used, as the FFSD and other datasets are medium-to-large scale, benefiting from more efficient batch processing. Additionally, a hidden layer dimension of 256 was consistently employed in the network to learn deep feature representations. A smaller learning rate of $\eta = 0.002$ was adopted to allow for more fine-grained parameter learning. Weight decay was set to $\lambda = 1 \times 10^{-4}$, applying this regularization technique to prevent overfitting. The number of layers in the prediction model TGAT was set to 4. In Table **??** in the ablation section, a 4-layer configuration yielded better results. Furthermore, to prevent overfitting, the model included two dropout layers with rates $D_1 = 0.2$ and $D_2 = 0.1$.

For training the datasets and conducting comparative experiments, the FFSD dataset setup proposed by Xiang et al. was followed. The dataset splits for the training set, test set, and validation set were configured in Table 4.1. The maximum number of epochs was set to 15, with an early stopping mechanism activated after 10 epochs to prevent overfitting. As shown in Figure 4.2, the model that achieved the most stable performance on the validation set during training was saved for testing on the test set. Based

Dataset	FSSD		
	AUC	F1	AP
MCNN [22]	0.7414 ± 0.0075	0.6285 ± 0.0068	0.3126 ± 0.0059
STAN [14]	0.7468 ± 0.0013	0.6399 ± 0.0021	0.3201 ± 0.0019
STAGN [13]	0.7008 ± 0.0017	0.5437 ± 0.0025	0.2659 ± 0.0023
GADGNN [16]	0.7746 ± 0.0012	0.7136 ± 0.0017	0.5981 ± 0.0014
TGAT [69]	0.7940 ± 0.0009	0.7182 ± 0.0014	0.6174 ± 0.0015
Ma-CVAE + TGAT	**0.8159 ± 0.0008**	**0.7372 ± 0.0015**	**0.6498 ± 0.0009**

Table 4.2: Comparison of the performance on the FFSD dataset using AUC, F1, and AP metrics with the SOTA model TGAT[69] and other advanced models. The best performance values are highlighted in bold, and the variance is used to indicate the error in performance.

on the feature dimensions of the three datasets, the multi-head sizes were set to Yelp: 4, Amazon: 5, and FFSD: 9.

4.3 Detection Performance

To comprehensively compare the results of the proposed model in credit card fraud detection, this paper conducts comparative experiments with both previous and current SOTA models in CCFD tasks and anomaly detection tasks, using the original parameters as set in their respective papers. The experiments were conducted five times, with the average taken and the variance added to represent the fluctuations, as shown in Tables 4.2, 4.3, and 4.4. Additionally, a comparison with a Baseline GCN model on the Amazon and YelpChi datasets is presented in Table A.1. The specific settings are as follows:

- **MCNN.** [22] A CNN-based framework designed to detect credit card fraud by capturing intrinsic patterns from labeled data. Following Cheng et al., the learning rate was set to 0.001 with training conducted over 30 epochs.
- **STAN.** [14] A spatial-temporal attention-based neural network for detecting credit card fraud, using attention and 3D convolution mechanisms to model spatial and temporal behaviors. Following Zhang et al., the learning rate was set to 0.0001 with training conducted over 20 epochs, and the attention hidden dimension was set to 150.
- **STAGN.** [13] A spatial-temporal attention-based graph network designed for CCFD, leveraging graph neural networks to learn temporal and location-based features, followed by 3D convolution. Following Cheng et al., the learning rate was set to 0.02, with training conducted over 70 epochs, and the attention hidden dimension was set to 150.
- **RQGNN.** [16] The first spectral GNN for graph-level anomaly detection, leveraging the Rayleigh Quotient to explore spectral features of anomalous graphs. The learning rate was set to 0.002, with training conducted over 15 epochs, and the hidden dimension was set to 256.
- **TGAT.** [69] A semi-supervised graph neural network designed for fraud detection, leveraging a Gated Temporal Attention Network to model temporal transaction graphs and risk propagation, currently the SOTA for this task. Following Xiang et al., the learning rate was set to 0.001 with training conducted over 15 epochs, and the hidden dimension was set to 256.

4.3.1 Performance in FFSD Dataset

Compared to the Amazon and YelpChi datasets, the performance on the FFSD dataset is somewhat weaker. This is primarily due to the significantly larger volume of transaction data in FFSD, which is 40 times that of YelpChi and 152 times that of Amazon. The presence of a large amount of data with unknown labels poses a higher demand for feature learning by the model. In Table 4.2, the proposed Ma-CVAE+TGAT model outperforms the SOTA model TGAT [69], achieving 0.8159, 0.7372, and 0.6498 in AUC, F1, and AP, respectively, with improvements of 2.75%, 2.65%, and 5.24% in these three metrics. The significant improvement in AP indicates excellent classification accuracy on minority samples. Similarly, compared to the previous SOTA model MCNN [22], which has a variance difference of 0.0075,

the Ma-CVAE+TGAT model only shows a variance of 0.0008 in the F1 metric, demonstrating greater stability.

4.3.2 Performance in Amazon Dataset

In Table 4.3, the proposed model demonstrates superior performance on the Amazon dataset compared to the other two datasets. This is primarily because the anomaly rate (6.87%,) in the Amazon dataset, which is the proportion of Fraud labels, the lowest among the datasets. This suggests that both current and previous SOTA models tend to perform better on Legitimate labels (majority class samples). The Ma-CVAE+TGAT model achieves the highest performance on the Amazon dataset, with an AUC of 0.9665, surpassing the SOTA model TGAT [69] by 1.52%. Additionally, it outperforms the SOTA model by 0.52% in F1 and 3.25% in AP, with the most significant improvement observed in the AP metric. The model's overall variance fluctuation, around 10^{-4}, indicates greater stability compared to the STAGN model [13].

Method	**Amazon**		
	AUC	**F1**	**AP**
MCNN [22]	0.7206 ± 0.0012	0.6914 ± 0.0014	0.4608 ± 0.0016
STAN [14]	0.8427 ± 0.0009	0.7993 ± 0.0011	0.6987 ± 0.0013
STAGN [13]	0.8414 ± 0.0010	0.7953 ± 0.0015	0.6395 ± 0.0012
GADGNN [16]	0.9261 ± 0.0008	0.8970 ± 0.0007	0.7997 ± 0.0009
TGAT [69]	0.9520 ± 0.0006	0.9150 ± 0.0004	0.8604 ± 0.0005
Ma-CVAE + TGAT	**0.9665 ± 0.0007**	**0.9198 ± 0.0002**	**0.8884 ± 0.0006**

Table 4.3: This table presents a comparison of various models' performance on the Amazon review dataset, evaluated using AUC, F1, and AP metrics. The TGAT[69] model, recognized as the SOTA, along with other advanced models, is included in this comparison. The best-performing results are emphasized in bold, and variance is provided to represent the performance error.

4.3.3 Performance in YelpChi Dataset

While the proposed Ma-CVAE model does not always outperform other models on the YelpChi dataset, it shows competitive results. Specifically, in terms of the F1 score, the Ma-CVAE model achieved 0.734, slightly behind the TGAT model [69] by 0.89%. However, the Ma-CVAE model maintained a leading advantage in AUC and AP metrics, surpassing the TGAT model by 2.44% and 3.47%, respectively. Additionally, the variance observed in the YelpChi dataset is slightly higher, reaching 0.0059 in the F1 score. This is primarily due to the difference in feature distribution between the YelpChi dataset and the Amazon and FFSD datasets. The latter datasets exhibit more discrete distributions, making the Gumbel distribution [33] more suitable, while the former has stronger correlations and continuity between features, making the Gaussian distribution more appropriate. As shown in Figure 3.5, even when a larger τ is used in the Gumbel distribution to create a smoother distribution, the performance still does not match that of the Gaussian distribution. In Section 4.8, the impact of these two distributions is further investigated.

Method	**YelpChi**		
	AUC	**F1**	**AP**
MCNN [22]	0.7903 ± 0.0094	0.6915 ± 0.0152	0.5293 ± 0.069
STAN [14]	0.8520 ± 0.0051	0.7290 ± 0.0031	0.6209 ± 0.0047
STAGN [13]	0.8432 ± 0.0011	0.7198 ± 0.0026	0.6192 ± 0.0023
GADGNN [16]	0.8796 ± 0.0006	0.7484 ± 0.0033	0.6228 ± 0.0025
TGAT [69]	0.8920 ± 0.0007	**0.7405 ± 0.0029**	0.6601 ± 0.0021
Ma-CVAE + TGAT	**0.9164 ± 0.0003**	0.7340 ± 0.0059	**0.6830 ± 0.0043**

Table 4.4: Comparison of model performance on the YelpChi dataset, evaluated using AUC, F1, and AP metrics. The highest performance values are highlighted in bold, with the corresponding variance indicating the margin of error.

Number of Layers(L)	FSSD		
	AUC	F1	AP
L = 1	0.7935	0.7188	0.6178
L = 2	0.8170	0.7301	0.6489
L = 3	0.8198	0.7361	0.6545
L = 4	**0.8271**	**0.7496**	**0.6683**
L = 5	0.8234	0.7439	0.6585
L = 2 (Without Ma-CVAE)	0.7940	0.7182	0.6174
L = 4 (With Ma-CVAE)	**0.8271**	**0.7496**	**0.6683**

Table 4.5: Comparison of TGAT model performance across different layers (L = 1 to L = 5) on the FSSD dataset, evaluated using AUC, F1, and AP metrics. The best-performing results are highlighted in bold. The comparison between L = 2 without Ma-CVAE and L = 4 with Ma-CVAE demonstrates that incorporating Ma-CVAE allows the optimal number of layers to increase, improving overall performance and reducing oversmoothing.

4.3.4 Impact of the Number of Layers in TGAT

In Table 4.5, to better understand the impact of the number of Graph Attention layers in TGAT, an experiment was conducted. Due to the limitation of GPU resources, the study on the number of layers was constrained, with the Number of Layers L varying from 1 to 5. The purpose of this experiment is to investigate the influence of the depth of the TGAT model on its ability to learn temporal patterns in the graph. The number of layers determines the model's ability to capture both local and global temporal dependencies. Generally, deeper TGAT models exhibit improved feature learning from the training set. As shown in Table 4.5, when L increases from 1 to 4, the model's performance on the FFSD dataset improves overall. The AUC rises from 0.7935 at $L = 1$ to a peak of 0.8271 at $L = 4$, an increase of 0.0336. The improvements in AUC, F1, and AP are 4.23%, 4.28%, and 8.17% respectively, indicating that the model's depth is highly sensitive to the number of layers.

However, as the number of layers increases further to $L = 5$, the model's performance on the FFSD dataset begins to decline, with decreases of 0.4474%, 0.7606%, and 1.4676% in the three metrics. This decline is primarily due to the over-smoothing problem in GNN-based models as the number of layers increases. Over-smoothing occurs when, with more layers, the features of different nodes become indistinguishable as they converge to similar values [52, 23]. This manifests as a decline in performance once the number of layers exceeds a certain threshold. The Ma-CVAE model proposed in this paper partially mitigates the feature convergence issue in the over-smoothing process by employing Gumbel distribution for feature sampling. In Table 4.5, the analysis comparing L = 2 without Ma-CVAE to L = 4 with Ma-CVAE indicates that including Ma-CVAE supports the use of more layers, which in turn boosts overall performance and minimizes oversmoothing, thereby enhancing the potential expressiveness of features.

4.4 Training Performance Analysis Across Datasets

To investigate the behavior of gradient updates during the training process, the trends in loss for both the training and validation sets are visualized, along with the changes in three metrics—AUC, AP, and Accuracy—across the three datasets FFSD, Amazon, and YelpChi over 15 epochs.

4.4.1 Training Loss Trends

As shown in Figure 4.2, the training loss exhibits an overall downward trend across all three datasets. Notably, the YelpChi and Amazon datasets display a distinct "elbow" pattern, characterized by a rapid initial decline followed by a more gradual decrease. In contrast, the FFSD dataset shows a slower rate of decline, primarily due to the larger data volume in FFSD, which is a significant percentage of the data in Amazon and YelpChi combined. Additionally, the FFSD dataset contains a substantial amount of data with unknown labels in Figure 4.1, requiring the model to spend considerable time learning the parameters, resulting in larger error bands compared to the other datasets.

For the validation set, the FFSD dataset shows a general downward trend as the number of epochs increases, with room for further improvement even after 15 epochs. The YelpChi dataset, however, begins to converge after 8 epochs, stabilizing around a loss of 0.003. The Amazon dataset, on the other hand,

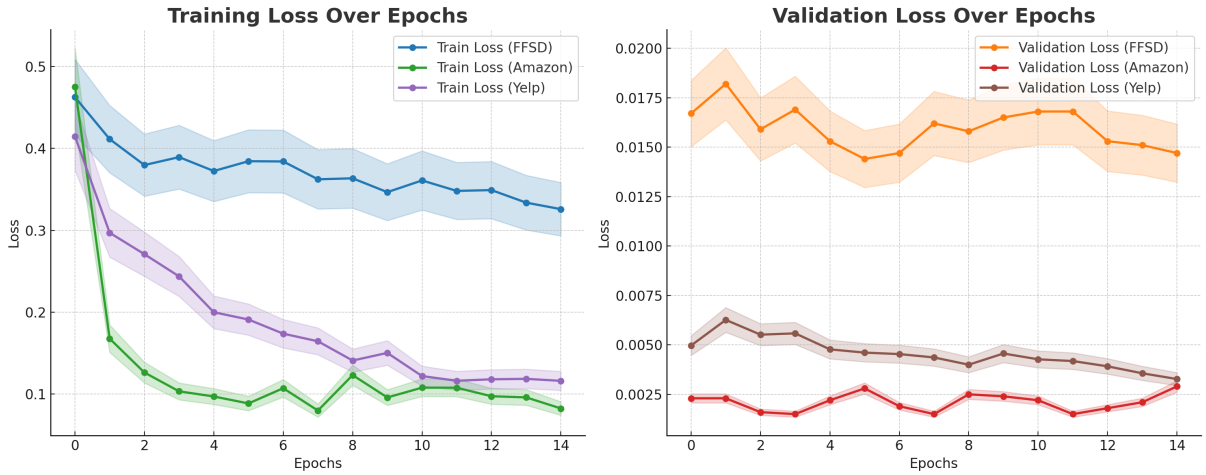


Figure 4.2: Illustration of the training and validation losses across different epochs for the FFSD [69], Amazon [34], and Yelp [49] datasets. The blue, green, and purple lines represent the training loss trends for the three datasets, showing a decreasing pattern as the number of epochs increases, indicating that the model continues to learn and adjust its parameters. The orange, brown, and red lines represent the validation losses. Unlike the training set, the validation set is unseen during training and is used to evaluate the model's performance on unseen data. For the Amazon dataset, an overfitting trend is observed as the number of epochs increases, which is why early stopping [45] is employed.

experiences fluctuations throughout the training process, with the validation loss reaching its lowest point at the 3rd and 7th epochs. Despite the training loss continuing to decrease, the validation loss begins to increase, indicating overfitting. To address this, an early stopping mechanism was introduced. This mechanism triggers if the model's performance on the validation set does not improve after a certain number of epochs, thereby automatically determining the optimal training duration.

4.4.2 Training Evaluation Metrics Trends

Due to the extreme label imbalance across the FFSD, Amazon, and YelpChi datasets, which is typical in CCFD tasks since anomalous events are generally rare, the loss function tends to be influenced by the majority class, potentially failing to adequately reflect the performance on the minority class.To address this, the trends of AUC, AP, and Accuracy over 15 epochs were visualized.

In Figure 4.3, the cyan lines across the three datasets exhibit an overall upward trend. For the FFSD dataset, all three metrics—AUC, AP, and Accuracy—increase with the number of epochs, indicating that the model is learning effectively from both the majority (legitimate) and minority (fraud) classes. However, it is noteworthy that there are significant drops in AP and AUC around epochs 11 and 14, suggesting that the model's accuracy in classifying minority samples decreased during these rounds, as the Accuracy metric did not decrease as much.

For the Amazon dataset, the metrics show a noticeable improvement as early as the first epoch, indicating that the features of the Amazon dataset are highly distinguishable, allowing the model to quickly learn useful information. However, as the number of training epochs increases, the AP metric exhibits strong fluctuations and is strongly correlated with AUC, indicating that the model's performance on the minority class (fraud) deteriorates at higher epochs, suggesting overfitting. An early stopping mechanism could be employed to mitigate this issue.

In contrast, the YelpChi dataset shows significant improvement only after the 9th epoch, suggesting that, compared to the Amazon dataset, the underlying features in the YelpChi dataset are less distinguishable, making it difficult for the model to learn meaningful representations in the early epochs. This indicates that the model was underfitting before epoch 8. Afterward, the model's performance on YelpChi stabilizes as it approaches convergence.

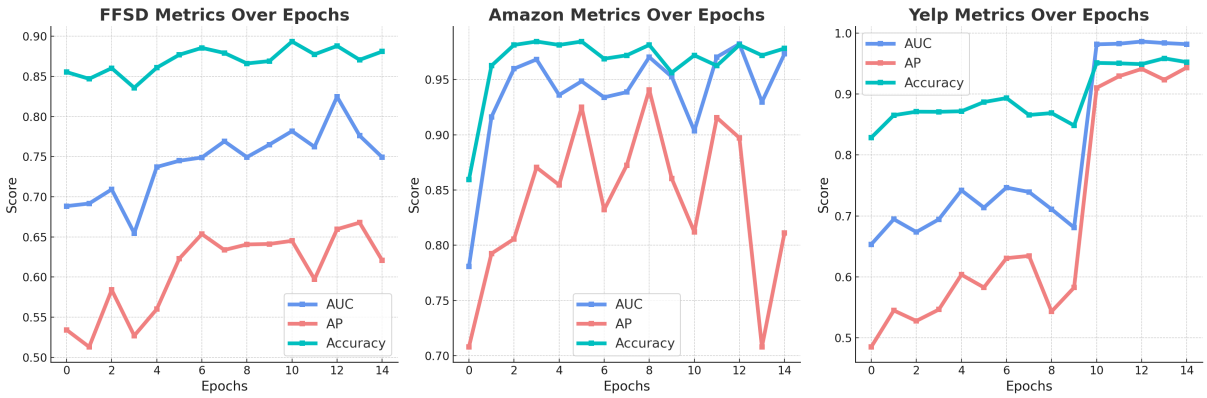


Figure 4.3: Illustration of the training performance using three metrics across 15 epochs for the FFSD, Amazon, and Yelp datasets. The cyan, blue, and red lines represent the trends of AUC, AP, and Accuracy for the three datasets.

4.5 Performance Comparison of Feature Processing Models

As mentioned in the Introduction, the increase in layers in GNN-based models can lead to feature homogenization, which adversely affects the accuracy of risk prediction. To investigate the impact of the proposed Ma-CVAE model and compare it with other mainstream generative models and feature processing models—such as the Variational AutoEncoder (VAE) [36], Conditional Diffusion [73], AutoEncoder (AE) [29], and Convolutional AutoEncoder (ConvAE) [63]—on the TGAT model's over-smoothing issue, experiments were designed, and the results are presented in Table 4.6. The table shows the comparison across three datasets: FSSD, Amazon, and YelpChi, using three evaluation metrics: AUC, F1, and AP.

The Ma-CVAE model overall achieved the best performance across all three datasets. Specifically, on the FSSD dataset, Ma-CVAE outperformed the AE [29] model by 0.0749 in AUC, representing an improvement rate of 10.17%. This indicates that the model performs well across both positive and negative samples. In terms of F1 and AP, which are metrics particularly relevant for imbalanced datasets, the improvement rates on the FSSD dataset reached 60.2% and 31.93%, respectively, demonstrating the model's strong performance in handling minority class samples.

In this experiment, the risk information from historical transactions, which serves as the label information, was used to supervise the model's learning. Notably, when Conditional supervision was applied, the AUC, F1, and AP metrics for AE [29] and ConvAE [63] on the FSSD dataset increased to 0.7469 and 0.5005, showing improvements of 1.40% and 2.65%, respectively. Ma-CVAE further optimized the Conditional supervision by incorporating a multi-head attention mechanism to adaptively focus on the label information, particularly emphasizing high-risk transaction labels and their associated neighboring transaction features.

In order to further explore the impact of Conditional supervision, this study improved upon the traditional VAE by using the same Gumbel distribution sampling technique [33] in Ma-CVAE. This approach helps to avoid the issue of poor fitting for discretized data during Gaussian distribution fitting, which can lead to a decline in the quality of generated data and potential gradient propagation issues. Compared to VAE [36], the proposed Ma-CVAE achieved an AUC improvement of 0.0046 on the financial dataset FSSD, and an AUC improvement of 0.0073 and 0.0567 on the two review datasets, Amazon and YelpChi, respectively. However, it is worth noting that VAE [36] exhibited better F1 performance on the FSSD and Amazon datasets, indicating that VAE [36] may be more effective in capturing features of minority samples in certain scenarios.

Furthermore, the experiment also explored the performance of diffusion models using noisy data [73], where the results were less favorable than those of the VAE [36] model. On the FSSD dataset, the diffusion model only achieved an AUC of 0.7929. This may be due to the distinct distribution patterns in the datasets, where noisy data did not fit well, even under Conditional supervision.

4.6 K-Fold Cross-Validation Analysis

To prevent the model from showing biased performance due to learning specific data features, K-Fold Cross-Validation was employed with K=5. In this approach, the dataset is divided into K folds, and

Dataset	FSSD			Amazon			YelpChi		
	AUC	F1	AP	AUC	F1	AP	AUC	F1	AP
AE [29]	0.7366	0.4513	0.4876	0.9200	0.9091	0.8081	0.8823	0.7429	0.6472
ConvAE [63]	0.7469	0.4513	0.5005	0.9559	0.9189	0.8682	0.8701	0.7034	0.6120
VAE [36]	0.8069	**0.7325**	0.6357	0.9592	**0.9206**	0.8724	0.8687	0.7174	0.6048
Con-Diffusion [73]	0.7929	0.7277	0.6268	0.9559	0.9189	0.8682	0.8765	0.7264	0.6244
Ma-CVAE	**0.8115**	0.7230	**0.6433**	**0.9665**	0.9198	**0.8884**	**0.9291**	**0.7928**	**0.7122**

Table 4.6: Comparison of the proposed Ma-CVAE model with other feature processing models, such as the generative model VAE[36] and the diffusion model Con-Diffusion[73], across three datasets: FSSD, Amazon, and YelpChi. The evaluation metrics used are AUC, F1, and AP, with the best values highlighted in bold.

during each iteration, the i-th fold is used as the validation set while the remaining $K-1$ folds are combined to form the training set. This ensures that each subset of the data has a chance to serve as the validation set, making the model evaluation more objective.

In Figure 4.4, the validation loss across 5 folds for a single epoch is illustrated. The results demonstrate that the model's performance varies across different subsets of the data. Notably, the FFSD dataset exhibits significant variation, with Fold 3 having the lowest validation loss of approximately 0.01412, while Fold 1 shows the highest loss at 0.01525, indicating more randomness in the results. In contrast, the Amazon and YelpChi datasets have lower overall losses across all 5 folds, with less variation between folds compared to the FFSD dataset. This difference is primarily due to the large amount of unknown data features in the FFSD dataset, which imposes a higher learning cost on the model. If a fold contains more Legitimate or Fraud labels, it provides more training information, leading to better performance in that specific fold. Therefore, the higher variability in the FFSD dataset can be attributed to these factors. Implementing the K-Fold Cross-Validation mechanism allows for a more accurate reflection of the model's performance on new data by averaging the results over multiple validations.

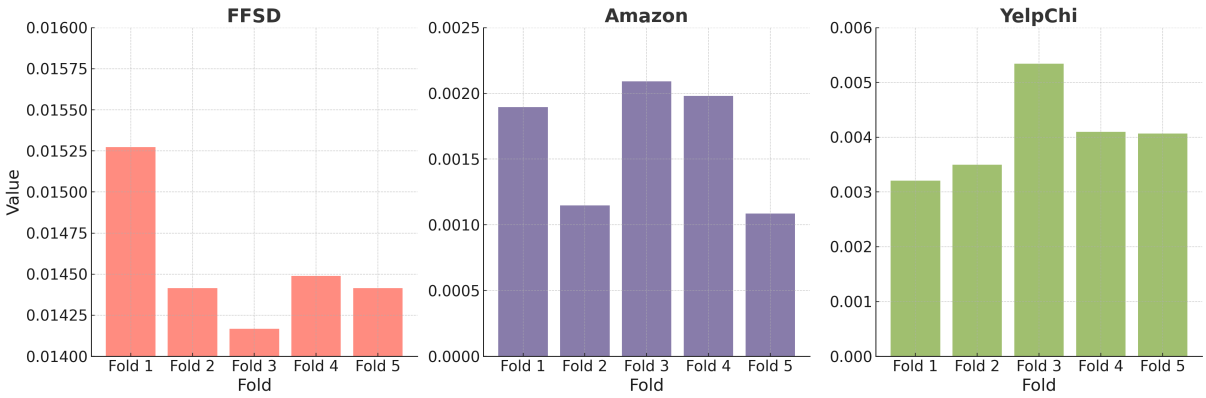


Figure 4.4: An illustration of the K-fold cross-validation process for the FFSD, Amazon, and Yelp datasets, showing the performance of the models on the validation set. To mitigate the instability in model evaluation caused by the randomness of the training data sampling, a smaller K value (K=5) is used, ensuring that each sample is utilized for both training and validation. Additionally, the validation losses across different folds demonstrate the specific performance variations across these folds.

4.7 Correlation Analysis of Time Series Features

As described in Section 3.1, the task of the experiment is to predict the risk level of the latest transaction based on the historical sequence of transaction features. The historical features used for training encompass temporal characteristics, where historical transaction data with high-risk features receive higher attention during model training, as well as greater focus on adjacent frames.

As shown in Figure 4.5, a correlation matrix was used to display the temporal correlations among historical transaction features. The matrix $\mathbf{X}_{\text{temp}} \in \mathbb{R}^{n \times d}$ was constructed by randomly selecting 30 con-

secutive historical transactions. A dynamic MLP [71] was applied to map the spatial feature dimensions to $\mathbb{R}^{n\times n}$ in Figure A.1. A heatmap was then used to represent these 30 transaction records, where a higher color intensity indicates a stronger correlation. It can be observed that the diagonal of the matrix represents the highest correlation between each feature and itself.

For the Amazon and FFSD datasets, the majority of adjacent transaction features do not exhibit high correlations. Only a few features, highlighted by red boxes, show clustered correlations. This is in contrast to the YelpChi dataset, where most of the 30 transactions display higher overall correlation. The primary reason for this difference is that the feature distributions in the Amazon and FFSD datasets are more dispersed. Apart from the highly correlated transaction features, most features do not exhibit significant correlations. This justifies the use of the Gumbel distribution, as defined in Eq. 3.15, for discrete data sampling in this paper, instead of the Gaussian distribution, which is not suitable for fitting the distribution of discrete data and can lead to reconstruction failure. In contrast, the YelpChi dataset exhibits correlations between various transactions, which aligns with the experimental results where Gaussian distribution can effectively prevent reconstruction failures. This is attributed to the high correlation among the features of most transactions.

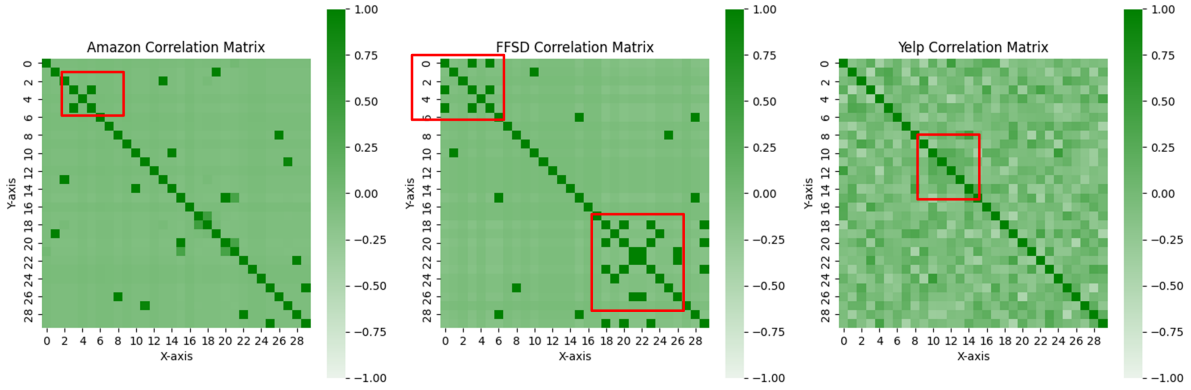


Figure 4.5: An illustration of the correlation matrix of historical time series features across three datasets: FFSD, Amazon, and Yelp. The time series features with high correlation are highlighted in red boxes. It can be observed from the figure that the deeper the green color, the higher the correlation between different time features. Apart from the diagonal, where the correlation between each feature and itself is the highest, there are highly correlated regions around key transaction features, forming clusters of high correlation. These clusters are useful for studying the correlation among time features. Additionally, the overall color intensity variations across different datasets reflect the differences in the distribution of features in these datasets.

4.8 Comparison of Distribution Methods in Ma-CVAE

To learn the latent representation of features, the model employs a technique that encodes features into a latent space and uses a specific distribution to fit these features [50, 36, 37]. However, the traditional VAE approach, which uses Gaussian distribution for resampling, is not well-suited for discrete data. In Figure 3.1, categorical attributes and numerical attributes are combined and input into the model. When these combined features exhibit discrete characteristics, forcing a Gaussian distribution to fit this data can lead to mismatches in the latent variable distribution, causing issues during training.

Experiments attempted to use Gaussian distribution sampling on three datasets: FFSD, Amazon, and YelpChi. However, due to the discrete nature of features in the FFSD and Amazon datasets, gradient propagation issues were encountered during the resampling process. In Figure 4.5, the FFSD and Amazon datasets exhibit significantly greater feature variability compared to the YelpChi dataset. Inspired by the work of Jang et al. [33], the Gumbel-Softmax method was adopted to handle the sampling process for these datasets with discrete features.

However, for the YelpChi dataset, which exhibits more continuous characteristics, the Ma-CVAE(Gumbel) method underperformed compared to the Ma-CVAE(Gaussian) method, lagging by approximately 3%. This indicates that the Gaussian distribution is better suited for handling continuous features in Table 4.7. Nevertheless, Ma-CVAE(Gumbel) still showed a 2.73% improvement over the baseline model without Ma-CVAE, demonstrating its effectiveness in enhancing feature representation.

Due to the gradient propagation issues observed with Ma-CVAE(Gaussian) on the Amazon and FFSD datasets, and to ensure a fair comparison with other methods, all experiments reported in Tables 4.2, 4.3, and 4.4 utilize the Ma-CVAE(Gumbel) method for consistency across the three datasets.

Dataset	YelpChi		
	AUC	F1	AP
No Ma-CVAE	0.8920	0.7405	0.6601
Ma-CVAE(Gumbel)	0.9164	0.7340	0.6830
Ma-CVAE(Gaussian)	**0.9448**	**0.7739**	**0.7670**

Table 4.7: Comparison of the performance on the YelpChi dataset without Ma-CVAE and with Ma-CVAE using two different sampling methods: Gumbel [33] and Gaussian distributions. The evaluation is based on three metrics: AUC, F1, and AP, with the best-performing results highlighted in bold.

4.9 Ablation Study

The proposed model in this paper primarily consists of two components: Ma-CVAE for enhancing latent feature representation and TGAT for feature extraction during the prediction phase. To better understand the contribution of Ma-CVAE and the impact of the number of layers in the TGAT network on performance, two experiments were designed to independently study these aspects.

4.9.1 Effectiveness of Ma-CVAE

Table 4.8 demonstrates the effectiveness of the Ma-CVAE model across three datasets through ablation experiments, where **w/o** and **w** indicate results without and with the Ma-CVAE model, respectively. Overall, the results show that incorporating the Ma-CVAE model significantly improves TGAT's classification and prediction capabilities, with better performance across all AUC, F1, and AP metrics. Specifically, on the financial dataset FFSD, the model achieved AUC, F1, and AP scores of 0.8115, 0.7230, and 0.6433, respectively, representing improvement rates of 2.20%, 0.67%, and 4.20%. Similarly, there were notable improvements on the two review datasets, with the AUC on the Amazon dataset increasing by 1.52% and the AP score by 3.25%. The YelpChi dataset saw the largest improvements, with AUC, F1, and AP increasing by 4.16%, 7.06%, and 7.89%, respectively.

Dataset	FSSD			Amazon			YelpChi		
	AUC	F1	AP	AUC	F1	AP	AUC	F1	AP
w/o	0.7940	0.7182	0.6174	0.9520	0.9150	0.8604	0.8920	0.7405	0.6601
w	**0.8115**	**0.7230**	**0.6433**	**0.9665**	**0.9198**	**0.8884**	**0.9291**	**0.7928**	**0.7122**

Table 4.8: Comparison of the performance across FSSD, Amazon, and YelpChi datasets without Ma-CVAE (w/o) and with Ma-CVAE (w). The evaluation is based on three metrics: AUC, F1, and AP, with the best-performing results highlighted in bold.

These results highlight the varying characteristics of the datasets. The Ma-CVAE model performed particularly well on the YelpChi dataset. Initially, a conventional Gaussian distribution-based approach [31] was used for feature sampling, which also yielded good results on YelpChi. However, due to the more discrete nature of the features in the FFSD and Amazon datasets, it was challenging to fit the sampled data into a Gaussian distribution, leading to gradient explosions during backpropagation. Although the use of a Gumbel distribution [33] mitigated this issue, the feature diversity and dispersion in these discrete datasets did not achieve the same effectiveness as with YelpChi. Furthermore, the overall greater improvement in F1 and AP compared to AUC suggests that the proposed model performs better on imbalanced datasets, indicating its superior performance in scenarios with limited samples.

Chapter 5

Conclusion

5.1 Contributions and Achievements

In recent years, with the advancement of technology, fraud transactions have become increasingly sophisticated, making it more challenging to distinguish them from legitimate transactions. This growing similarity has led to inaccuracies in predictive risk assessments for fraud transactions that closely resemble legitimate ones. Although GNN-based methods [69, 16, 13, 42, 13] have been proposed to leverage graph structures for learning the relationships between different transactions, these methods still struggle to effectively identify fraud transactions that are similar to legitimate ones. Furthermore, as the number of layers in GNN-based models increases, the model may become overly smooth, leading to a homogenization of features between legitimate and fraud transactions, which diminishes the model's ability to distinguish between them [52, 23].

This work introduces a novel Multi-head Attention Conditional Variational Autoencoder technique that enhances the diversity of features between legitimate and fraud transactions, thereby improving the model's ability to distinguish between different classes and increasing the accuracy of risk predictions.

This work preprocesses the data using a time window T, where each transaction within this window is processed to generate a series of features in Table 3.1. By increasing the feature information entropy [22], which is conducive to model differentiation and learning risk information, the new features are combined with the original numerical features for training. Additionally, a temporal graph is constructed using a specialized graph construction library based on the transaction order over time for learning in the TGAT model. Moreover, a Random Label Model is designed to randomly mask the model's labels, thereby enhancing the robustness of the training process.

Then, the processed data features are input into the proposed Ma-CVAE model for feature enhancement. Specifically, the multi-head attention mechanism is employed to adaptively learn the key label information that plays a crucial role in risk prediction. The processed label information is then used as a training condition to supervise the learning process, where the numerical and categorical features are mapped into a Gumbel-Softmax distribution space to learn potential feature variations. This guides the diversification of features, resulting in a more diverse reconstructed dataset. This dataset, combined with the temporal graph, serves as the input for an L-layer Temporal Graph Attention Network, where multi-head attention is used to aggregate information from neighboring nodes, enabling more accurate learning of neighbor information and model propagation. The final risk prediction result is obtained by mapping the output through a two-layer MLP to the range [0,1].

This work conducts comparative experiments across three datasets against five state-of-the-art models. The proposed model achieves superior performance on all three unsupervised datasets. Specifically, on the financial dataset FFSD, the proposed model outperforms the SOTA TGAT model with improvements of 2.75%, 2.65%, and 5.24% in AUC, F1, and AP, respectively. On the two review datasets, Amazon and YelpChi, the model achieves AUC improvements of 1.52% and 2.44% over the SOTA models. Additionally, this work examines the impact of varying the number of TGAT layers L to mitigate the oversmoothing issue in deep GNN-based models. Furthermore, the Ma-CVAE model is compared against other data augmentation and generative models, including AE [29], Conv-AE [63], VAE [36], and Con-Diffusion [73]. The results demonstrate the superiority of the Ma-CVAE model, with an average improvement of 2.62% in AUC across the three datasets, underscoring the effectiveness of the proposed approach.

5.2 Future Work

Future work will continue to focus on improving predictive models for CCFD. The core of the proposed Ma-CVAE model involves enhancing feature diversity through distribution-based expansion and supervised learning with multi-head attention mechanisms.

One of the key components of the Ma-CVAE model is its multi-head attention mechanism. This mechanism computes self-attention [65] weights, which are applied to the target feature matrix. This approach allows the model to concentrate on the most significant features while disregarding less relevant ones. In Figure 5.1, in a standard attention mechanism [65], the intensity of the attention weights is represented by color, with higher weights indicating a greater focus on specific features.

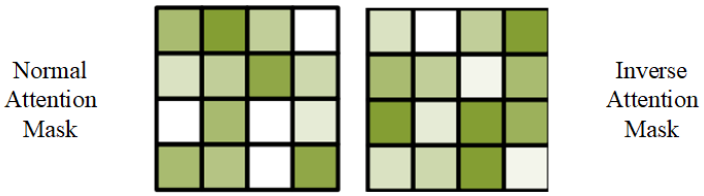


Figure 5.1: Illustration of the complementary attention weight matrix. The complementary attention mechanism consists of Normal Attention and Inverse Attention, where the value of Attention is denoted as $\mathbf{A}$, and the value of inverse Attention can be expressed as $1 - \mathbf{A}$. A learnable weight $\mathbf{W_a}$ is used to balance these two types of Attention, resulting in a reasonable weight distribution.

However, as highlighted by Duan et al. [18], traditional attention mechanisms face challenges related to attention bias. Specifically, these mechanisms tend to disproportionately focus on a majority of features that appear important, while neglecting the influence of less prominent features. In the context of CCFD tasks, this often results in an overemphasis on transaction labels associated with mainstream consumption patterns and habits. Although this focus might seem intuitive, it can cause the model to overlook less obvious but equally crucial features related to more diverse user transactions. This oversight is particularly problematic because these less common features, despite representing a minority, can collectively have a significant impact on the model's overall performance—a phenomenon referred to as the "Long Tail Effect" [3, 62]. Addressing this bias is essential to ensure that the model accurately captures the full spectrum of transaction behaviors, thereby improving its ability to detect fraud across a wider range of scenarios.

To address this challenge and balance the focus between prevalent features and rare but important features, future work will explore methods to mitigate attention bias. Following the approach suggested by Duan et al. [18], one possible solution is to develop an inverse attention mechanism, as illustrated in Figure 5.1. This involves setting an adaptive weight parameter to balance the training of both attention matrices. By adjusting these weights based on loss backpropagation, it is possible to find an optimal balance that addresses the issue of attention bias and improves model performance.

Bibliography

[1] Ayman Alazizi, Amaury Habrard, François Jacquenet, Liyun He-Guelton, and Frédéric Oblé. Dual sequential variational autoencoders for fraud detection. In *Advances in Intelligent Data Analysis XVIII: 18th International Symposium on Intelligent Data Analysis, IDA 2020, Konstanz, Germany, April 27–29, 2020, Proceedings 18*, pages 14–26. Springer, 2020.

[2] Latifa AlFalahi and Haitham Nobanee. Conceptual building of sustainable economic growth and corporate bankruptcy. *Available at SSRN 3472409*, 2019.

[3] Chris Anderson. *The Long Tail: Why the Future of Business is Selling Less of More*. Hyperion, 2006.

[4] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. Statistical inference for probabilistic functions of finite state markov chains. *The Annals of Mathematical Statistics*, 37(6):1554–1563, 1966.

[5] Ibtissam Benchaji, Samira Douzi, Bouabid El Ouahidi, and Jaafar Jaafari. Enhanced credit card fraud detection based on attention mechanism and lstm deep model. *Journal of Big Data*, 8:1–21, 2021.

[6] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.

[7] Siddhartha Bhattacharyya, Sanjeev Jha, Kurian Tharakunnel, and J Christopher Westland. Data mining for credit card fraud: A comparative study. *Decision support systems*, 50(3):602–613, 2011.

[8] Rejwan Bin Sulaiman, Vitaly Schetinin, and Paul Sant. Review of machine learning approach on credit card fraud detection. *Human-Centric Intelligent Systems*, 2(1):55–68, 2022.

[9] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[10] Nitesh V. Chawla, Nathalie Japkowicz, and Aleksander Kotcz. Learning from imbalanced data. *ACM SIGKDD Explorations Newsletter*, 6(1):1–6, 2004.

[11] Joy Iong-Zong Chen and Kong-Long Lai. Deep convolution neural network model for credit-card fraud detection and alert. *Journal of Artificial Intelligence*, 3(02):101–112, 2021.

[12] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

[13] Dawei Cheng, Xiaoyang Wang, Ying Zhang, and Liqing Zhang. Graph neural network for fraud detection via spatial-temporal attention. *IEEE Transactions on Knowledge and Data Engineering*, 34(8):3800–3813, 2020.

[14] Dawei Cheng, Sheng Xiang, Chencheng Shang, Yiyi Zhang, Fangzhou Yang, and Liqing Zhang. Spatio-temporal attention-based neural network for credit card fraud detection. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 362–369, 2020.

[15] Alberto Dal Pozzolo, Olivier Caelen, Yann-Aël Le Borgne, Serge Waterschoot, and Gianluca Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE transactions on neural networks and learning systems*, 27(8):1710–1723, 2015.

[16] Xiangyu Dong, Xingyi Zhang, and Sibo Wang. Rayleigh quotient graph neural networks for graph-level anomaly detection. In *Proceedings of the International Conference on Learning Representations, ICLR*, 2024.

[17] Yingtong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM international conference on information & knowledge management*, pages 315–324, 2020.

[18] Jinghai Duan, Le Wang, Chengjiang Long, Sanping Zhou, Fang Zheng, Liushuai Shi, and Gang Hua. Complementary attention gated network for pedestrian trajectory prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 542–550, 2022.

[19] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pages 226–231, 1996.

[20] Hosein Fanai and Hossein Abbasimehr. A novel combined approach based on deep autoencoder and deep classifiers for credit card fraud detection. *Expert Systems with Applications*, 217:119562, 2023.

[21] Ugo Fiore, Alfredo De Santis, Francesca Perla, Paolo Zanetti, and Francesco Palmieri. Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. *Information Sciences*, 479:448–455, 2019.

[22] Kang Fu, Dawei Cheng, Yi Tu, and Liqing Zhang. Credit card fraud detection using convolutional neural networks. In *Neural Information Processing: 23rd International Conference, ICONIP 2016, Kyoto, Japan, October 16–21, 2016, Proceedings, Part III 23*, pages 483–490. Springer, 2016.

[23] Jhony H Giraldo, Konstantinos Skianis, Thierry Bouwmans, and Fragkiskos D Malliaros. On the trade-off between over-smoothing and over-squashing in deep graph neural networks. In *Proceedings of the 32nd ACM international conference on information and knowledge management*, pages 566–576, 2023.

[24] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, pages 2672–2680, 2014.

[25] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. Improved training of wasserstein gans. *Advances in Neural Information Processing Systems*, pages 5767–5777, 2017.

[26] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-smote: A new over-sampling method in imbalanced data sets learning. In *International Conference on Intelligent Computing*, pages 878–887. Springer, 2005.

[27] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, pages 1322–1328. IEEE, 2008.

[28] Irina Higgins, Loic Matthey, Arka Pal, Christopher P Burgess, Xavier Glorot, Matthew M Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. *ICLR*, 3, 2017.

[29] Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

[30] Rob J Hyndman, Anne B Koehler, Ralph D Snyder, and Simone Grose. Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association*, 97(460):1047–1062, 2002.

[31] Bekkouch Imad Ibrahim, Dragoş Constantin Nicolae, Adil Khan, Syed Imran Ali, and Asad Khattak. Vae-gan based zero-shot outlier detection. In *Proceedings of the 2020 4th international symposium on computer science and intelligent control*, pages 1–5, 2020.

[32] Emmanuel Ileberi, Yanxia Sun, and Zenghui Wang. Performance evaluation of machine learning methods for credit card fraud detection using smote and adaboost. *IEEE Access*, 9:165286–165294, 2021.

[33] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

[34] Ananya Harsh Jha, Saket Anand, Maneesh Singh, and VS Rao Veeravasarapu. Disentangling factors of variation with cycle-consistent variational auto-encoders. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 805–820, 2018.

[35] Johannes Jurgovsky, Michael Granitzer, Konstantin Ziegler, Sylvie Calabretto, Pierre-Edouard Portier, Liyun He-Guelton, and Olivier Caelen. Sequence classification for credit-card fraud detection. *Expert systems with applications*, 100:234–245, 2018.

[36] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[37] Durk P Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. *Advances in neural information processing systems*, 28, 2015.

[38] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[39] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.

[40] Guojun Liang, Prayag Tiwari, Sławomir Nowaczyk, Stefan Byttner, and Fernando Alonso-Fernandez. Dynamic causal explanation based diffusion-variational graph neural network for spatiotemporal forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[41] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. Pick and choose: a gnn-based imbalanced learning approach for fraud detection. In *Proceedings of the web conference 2021*, pages 3168–3177, 2021.

[42] Zhiwei Liu, Yingtong Dou, Philip S Yu, Yutong Deng, and Hao Peng. Alleviating the inconsistency problem of applying graph neural network to fraud detection. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*, pages 1569–1572, 2020.

[43] Zhiwei Liu, Yingtong Dou, Philip S Yu, Yutong Deng, and Hao Peng. Alleviating the inconsistency problem of applying graph neural network to fraud detection. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*, pages 1569–1572, 2020.

[44] Domician Máté, Rabeea Sadaf, Judit Oláh, József Popp, and Edit Szűcs. The effects of accountability, governance capital, and legal origin on reported frauds. *Technological and Economic Development of Economy*, 25(6):1213–1231, 2019.

[45] Nelson Morgan and Hervé Bourlard. Generalization and parameter estimation in feedforward nets: Some experiments. *Advances in neural information processing systems*, 2, 1989.

[46] Naresh Babu Muppalaneni, Maode Ma, Sasikumar Gurumoorthy, P Ragha Vardhani, Y Indira Priyadarshini, and Y Narasimhulu. Cnn data mining algorithm for detecting credit card fraud. *Soft computing and medical bioinformatics*, pages 85–93, 2019.

[47] Teuku Rizky Noviandy, Ghalieb Mutig Idroes, Aga Maulana, Irsan Hardi, Edi Saputra Ringga, and Rinaldi Idroes. Credit card fraud detection for contemporary financial management using xgboost-driven machine learning and data augmentation techniques. *Indatu Journal of Management and Accounting*, 1(1):29–35, 2023.

[48] Apapan Pumsirirat and Yan Liu. Credit card fraud detection using deep learning based on auto-encoder and restricted boltzmann machine. *International Journal of advanced computer science and applications*, 9(1), 2018.

[49] Shebuti Rayana and Leman Akoglu. Collective opinion spam detection: Bridging review networks and metadata. In *Proceedings of the 21th acm sigkdd international conference on knowledge discovery and data mining*, pages 985–994, 2015.

[50] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *International conference on machine learning*, pages 1278–1286. PMLR, 2014.

[51] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[52] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993*, 2023.

[53] Y Sahin and Ekrem Duman. Detecting credit card fraud by decision trees and support vector machines. In *Proceedings of the International MultiConference of Engineers and Computer Scientists*, volume 1, pages 1–6, 2011.

[54] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

[55] KR Seeja and Masoumeh Zareapoor. Fraudminer: A novel credit card fraud detection model based on frequent itemset mining. *The Scientific World Journal*, 2014(1):252797, 2014.

[56] Xu Shen, Pietro Lio, Lintao Yang, Ru Yuan, Yuyang Zhang, and Chengbin Peng. Graph rewiring and preprocessing for graph neural networks based on effective resistance. *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[57] Fengzhao Shi, Yanan Cao, Yanmin Shang, Yuchen Zhou, Chuan Zhou, and Jia Wu. H2-fdetector: A gnn-based fraud detector with homophilic and heterophilic connections. In *Proceedings of the ACM web conference 2022*, pages 1486–1494, 2022.

[58] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. Masked label prediction: Unified message passing model for semi-supervised classification. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020.*, pages 1548–1554. International Joint Conferences on Artificial Intelligence Organization, 2020.

[59] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

[60] Abhinav Srivastava, Amlan Kundu, Shamik Sural, and Arun Majumdar. Credit card fraud detection using hidden markov model. *IEEE Transactions on dependable and secure computing*, 5(1):37–48, 2008.

[61] Junyi Tang, Simin An, Yuanwei Liu, Yong Su, and Jin Chen. M2ast: Mlp-mixer-based adaptive spatial-temporal graph learning for human motion prediction. *Multimedia Systems*, 30(4):206, 2024.

[62] Junyi Tang, Jin Chen, Yong Su, Meng Xing, and Shuang Zhu. Mtan: Multi-degree tail-aware attention network for human motion prediction. *Internet of Things*, 25:101134, 2024.

[63] Markus Thill, Wolfgang Konen, Hao Wang, and Thomas Bäck. Temporal convolutional autoencoder for unsupervised anomaly detection in time series. *Applied Soft Computing*, 112:107751, 2021.

[64] Dedy Trisanto, Nofita Rismawati, Mulya Muhamad Femy, and Kurniadi Felix Indra. Modified focal loss in imbalanced xgboost for credit card fraud detection. *International Journal of Intelligent Engineering and Systems*, 14(4):350–358, 2021.

[65] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[66] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[67] Daixin Wang, Jianbin Lin, Peng Cui, Quanhui Jia, Zhen Wang, Yanming Fang, Quan Yu, Jun Zhou, Shuang Yang, and Yuan Qi. A semi-supervised graph attentive network for financial fraud detection. In *2019 IEEE international conference on data mining (ICDM)*, pages 598–607. IEEE, 2019.

[68] Yue Wu, Yunjie Xu, and Jiaoyang Li. Feature construction for fraudulent credit card cash-out detection. *Decision Support Systems*, 127:113155, 2019.

[69] Sheng Xiang, Mingzhi Zhu, Dawei Cheng, Enxia Li, Ruihui Zhao, Yi Ouyang, Ling Chen, and Yefeng Zheng. Semi-supervised credit card fraud detection via attribute-driven graph representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 14557–14565, 2023.

[70] Shiyang Xuan, Guanjun Liu, Zhenchuan Li, Lutao Zheng, Shuo Wang, and Changjun Jiang. Random forest for credit card fraud detection. In *2018 IEEE 15th international conference on networking, sensing and control (ICNSC)*, pages 1–6. IEEE, 2018.

[71] Lingfeng Yang, Xiang Li, Renjie Song, Borui Zhao, Juntian Tao, Shihao Zhou, Jiajun Liang, and Jian Yang. Dynamic mlp for fine-grained image classification by leveraging geographical and temporal information. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10945–10954, 2022.

[72] Mohamad Zamini and Gholamali Montazer. Credit card fraud detection using autoencoder based clustering. In *2018 9th International Symposium on Telecommunications (IST)*, pages 486–491. IEEE, 2018.

[73] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3836–3847, 2023.

[74] Muhan Zhang, Shali Jiang, Zhicheng Cui, Roman Garnett, and Yixin Chen. D-vae: A variational autoencoder for directed acyclic graphs. *Advances in neural information processing systems*, 32, 2019.

[75] Honghao Zhu, MengChu Zhou, Guanjun Liu, Yu Xie, Shijun Liu, and Cheng Guo. Nus: Noisy-sample-removed undersampling scheme for imbalanced classification and application to credit card fraud detection. *IEEE Transactions on Computational Social Systems*, 2023.

Appendix A

Appendix

A.1 Dynamic-MLP

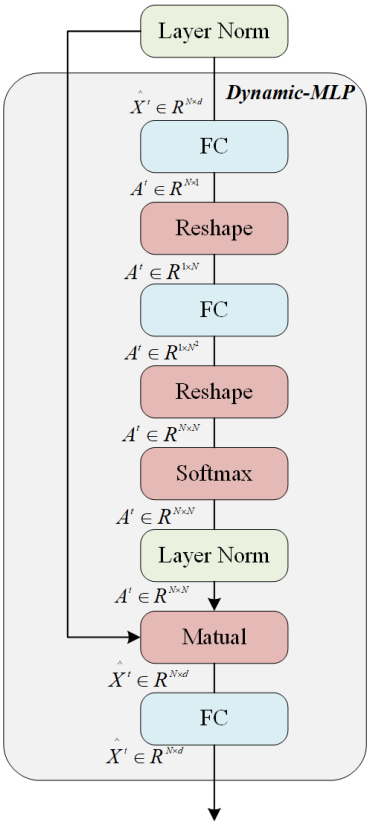


Figure A.1: An illustration of the dynamic MLP structure. This structure maps the given input matrix with dimensions $\mathbb{R}^{N\times d}$ to a weight matrix with dimensions $\mathbb{R}^{N\times N}$, where N represents the temporal dimension. The resulting temporal matrix captures the feature correlations along the temporal dimension. This feature correlation is then multiplied with the original input matrix to obtain a new matrix that integrates temporal feature correlations.

A.2 Comparison with Baseline GCN

Dataset	Amazon		YelpChi	
	AUC	**F1**	**AUC**	**F1**
GCN(Baseline)[41]	0.836	0.648	0.598	0.562
Ma-CVAE + TGAT	**0.966**	**0.919**	**0.929**	**0.792**

Table A.1: Comparison of the proposed Ma-CVAE + TGAT model with the traditional Baseline GCN [41] across the Amazon and YelpChi datasets based on two metrics: AUC and F1. The best-performing results are highlighted in bold.